

Міністерство освіти та науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Звіт до комп’ютерного практикуму №6

з дисципліни «Комп’ютерне моделювання дискретно-подійних систем»

Прийняв:
Викладач
Д.С. Коринецький

Виконав
Студент 4 курсу, гр. ІП-33
Ломакін Максим

Оцінка: _____

«___» _____ 2026 р.

Київ 2026

Застосування алгоритму стохастичної мережі Петрі для реалізації моделей дискретно-подійних систем

1. Текст завдання:

1. Ознайомитись з бібліотекою класів PetriObjModelPaint https://github.com/StetsenkoInna/PetriObjModelPaint моделювання дискретно-подійних систем на основі стохастичних мереж Петрі та графічним редактором мережі Петрі. З використанням алгоритму імітації стохастичної мережі Петрі класу PetriSim реалізувати модель, розроблену за текстом завдання 1 практикуму 5, та виконати її верифікацію. Зробити висновки про функціонування моделі. 20 балів.
2. З використанням алгоритму імітації стохастичної мережі Петрі класу PetriSim реалізувати модель, розроблену за текстом завдання 4 практикуму 5, та виконати її верифікацію. Зробити висновки про функціонування моделі. 20 балів.
3. Побудувати модель системи, що відтворює багатопотокову обробку запитів на кожному з двох серверів: головному та допоміжному. Для запобігання DDOS атакам для черги запитів встановлено обмеження до 1000. Результат обробки запиту обов’язково зберігається у базі даних перед його поверненням користувачу. База даних є спільною для обох серверів і забезпечена багатопотоковим доступом. Ймовірність звернення до допоміжного сервера 0,3. Часові характеристики обробки запитів, кількість обчислювального ресурсу, задіяного в обчисленнях, та кількість потоків, задіяних у програмі обчислень, задайте самостійно. Метою моделювання є оцінка часу обробки запиту. 20 балів.
4. Для усіх розроблених мереж Петрі створити та подати разом зі звітом такий пакет файлів: 1) текст завдання, 2) скриншот графічного зображення розробленої мережі Петрі у форматі .png, яка є результатом збереження Save GraphNet, 3) результат збереження мережі Петрі у форматі .pnml, 4) результат збереження для мережі Петрі у вигляді java-методу у форматі .txt. 20 балів.
5. Побудувати математичні рівняння, що описують побудовану за текстом завдання 3 мережу Петрі. 20 балів.

Задача 1 практикуму 5. Конвеєрна система складається з п'ятьох обслуговуючих пристроїв, розташованих уздовж стрічки конвеєра. Деталі надходять на опрацювання на перший пристрій із постійною швидкістю, рівної 4 одиниці за 1 хвилину. Тривалість обслуговування на кожному пристрої розподілена за експоненціальним законом з математичним сподіванням 1 хвилина. Вільного місця перед кожним конвеєром немає, тому пристрій може зняти деталь із конвеєра, тільки якщо знаходиться в стані «вільний». Якщо перший пристрій вільний, то деталь обробляється на ньому. По закінченні обробляння деталь залишає систему. Якщо перший пристрій зайнятий у момент надходження деталі, деталь по конвеєру надходить до другого пристрою. Інтервал проходження деталі між пристроями дорівнює 1 хвилина. Якщо при прямуванні деталі по конвеєру всі пристрої були зайняті, вона повертається до першого пристрою з затримкою 5 хвилин. Метою моделювання є визначення статистичних характеристик часу перебування деталі в системі, завантаження обслуговуючих пристроїв і кількості зайнятих пристроїв.

Задача 4 практикуму 5. У супермаркеті планується ввести систему управління запасами холодильників. Час між надходженнями замовлень на холодильники має експоненціальний розподіл з математичним сподіванням 0,2 тижні. Якщо покупцю знадобився холодильник тоді, коли його в запасі немає, він у 80% випадків відправляється в інший найближчий магазин, представляючи тим самим продаж, що не відбувся для даного універмагу. У 20% таких випадків робиться повторне замовлення, і покупці чекають надходження наступної партії вантажу. Магазин використовує періодичну систему перегляду стана запасів, у якому запас проглядається кожні 4 тижні і приймається рішення про необхідність здійснення замовлення. Стратегія прийняття рішення складається в розміщенні замовлення, що доводить запас до контрольного рівня, що складає 72 холодильники. Поточний стан запасу визначається як наявний запас плюс замовлені раніше приймачі і мінус невдоволений попит. Якщо поточний стан запасів менше або дорівнює 18 холодильникам (точка замовлення), здійснюється розміщення замовлення. Час доставки (час між розміщенням замовлення і його одержання) постійний і складає 3 тижні. Початкові умови: стан запасу – 72 холодильника, невдоволеного попиту немає. Визначити середню кількість холодильників у запасі, середній час між продажами, що не здійснилися.

2. Код програми:

Моделі написано мовою Java 23 з бібліотекою PetriObjModelPaint версії 2.3.0 (готова збірка графічного редактора містить і бібліотеку `petri-math` з класами `PetriP`, `PetriT`, `ArcIn`, `ArcOut`, `PetriNet`, `PetriSim`, і модуль `petri-model` з графічною мережею та форматом PNML). Мережі Петрі будуються з класів бібліотеки так само, як у методах класу `NetLibrary`, а імітація виконується методами класу `PetriSim`. Вихідний код, пакет файлів мереж і результати розміщено в репозиторії https://github.com/kqjqxb/Computer-simulation (каталог `lab6`). Файли програми:

- `NetBuilder.java` – побудова мережі Петрі з позицій, переходів, звичайних та інформаційних дуг разом з координатами елементів для графічного зображення;
- `Simulation.java` – імітація мережі алгоритмом класу `PetriSim`, статистика з урахуванням розгону, трасування подій і спостерігачі за спрацюванням переходів;
- `Conveyor.java`, `Inventory.java`, `Servers.java` – моделі завдань 1–3, їх перевірка та експерименти;
- `NetExport.java` – збереження мереж у форматах .pns, .png, .pnml і у вигляді java-методу засобами графічного редактора;
- `Queueing.java` – формули M/M/c для перевірки моделі серверів; `Checks.java`, `Stats.java`, `Json.java`, `Output.java`, `Main.java` – перевірки, довірчі інтервали, запис результатів і запуск;
- `compare_lab5.py` – порівняння структури мереж і результатів з практикумом 5; `plots.py` – графіки; `run.sh` – завантаження бібліотеки, компіляція і запуск.

Файл NetBuilder.java

```
package lab6;

import java.awt.geom.Point2D;
import java.util.ArrayList;
import java.util.IdentityHashMap;
import java.util.List;
import java.util.Map;

import ua.stetsenkoinna.petriobj.ArcIn;
import ua.stetsenkoinna.petriobj.ArcOut;
import ua.stetsenkoinna.petriobj.ExceptionInvalidTimeDelay;
import ua.stetsenkoinna.petriobj.PetriNet;
import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriT;

/**
 * Побудова мережі Петрі з класів бібліотеки PetriObjModelPaint (PetriP, PetriT, ArcIn, ArcOut)
 * разом з координатами елементів для графічного зображення мережі.
```

```
 */
public final class NetBuilder {

    private final String name;
    private final List<PetriP> places = new ArrayList<>();
    private final List<PetriT> transitions = new ArrayList<>();
    private final List<ArcIn> arcsIn = new ArrayList<>();
    private final List<ArcOut> arcsOut = new ArrayList<>();
    private final Map<Object, Point2D> layout = new IdentityHashMap<>();

    public NetBuilder(String name) {
        this.name = name;
        // номери позицій, переходів і дуг бібліотека бере з глобальних лічильників
        PetriP.initNext();
        PetriT.initNext();
        ArcIn.initNext();
        ArcOut.initNext();
    }

    public PetriP place(String placeName, int mark, double x, double y) {
        PetriP p = new PetriP(placeName, mark);
        places.add(p);
        layout.put(p, new Point2D.Double(x, y));
        return p;
    }

    /** Перехід зі сталою затримкою delay. */
    public PetriT transition(String transitionName, double delay, double x, double y) {
        PetriT t = new PetriT(transitionName, delay);
        transitions.add(t);
        layout.put(t, new Point2D.Double(x, y));
        return t;
    }

    /** Перехід з експоненційно розподіленою затримкою із середнім mean. */
    public PetriT expTransition(String transitionName, double mean, double x, double y) {
        PetriT t = transition(transitionName, mean, x, y);
        t.setDistribution("exp", mean);
        return t;
    }

    public void arc(PetriP p, PetriT t, int k) {
        arcsIn.add(new ArcIn(p, t, k));
    }

    public void arc(PetriP p, PetriT t) {
        arc(p, t, 1);
    }

    public void arc(PetriT t, PetriP p, int k) {
        arcsOut.add(new ArcOut(t, p, k));
    }

    public void arc(PetriT t, PetriP p) {
        arc(t, p, 1);
    }

    /** Інформаційна дуга: перевіряє наявність k маркерів у позиції, але не забирає їх. */
    public void info(PetriP p, PetriT t, int k) {
        arcsIn.add(new ArcIn(p, t, k, true));
    }

    public PetriNet build() {
        try {
            return new PetriNet(name, new ArrayList<>(places), new ArrayList<>(transitions),
                    new ArrayList<>(arcsIn), new ArrayList<>(arcsOut));
        } catch (ExceptionInvalidTimeDelay e) {
            throw new IllegalStateException(e);
        }
    }

    public String name() {
        return name;
    }

    public List<PetriP> places() {
        return places;
```

```
    }

    public List<PetriT> transitions() {
        return transitions;
    }

    public List<ArcIn> arcsIn() {
        return arcsIn;
    }

    public List<ArcOut> arcsOut() {
        return arcsOut;
    }

    public Point2D position(Object element) {
        return layout.get(element);
    }
}
```

Файл Simulation.java

```
package lab6;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.TreeMap;
import java.util.function.Consumer;
import java.util.function.DoubleConsumer;
import java.util.function.ToDoubleFunction;

import ua.stetsenkoinna.petriobj.FiringPhase;
import ua.stetsenkoinna.petriobj.PetriNet;
import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriSim;
import ua.stetsenkoinna.petriobj.PetriT;

/**
 * Імітація мережі Петрі алгоритмом класу PetriSim: input() – вхід маркерів у переходи з
 * розв'язанням конфліктів, getTimeMin() – момент найближчої події, output() – вихід маркерів з
 * усіх переходів, час виходу яких дорівнює поточному моменту. Порядок викликів той самий, що в
 * PetriObjModel.go() для одного Петрі-об'єкта. Додатково збираються статистики з урахуванням
 * періоду розгону, підраховуються входи й виходи переходів і виконується трасування подій.
 */
public final class Simulation {

    /** Середнє за часом довільної функції стану мережі та її розподіл за часом. */
    public static final class Watch {
        private final ToDoubleFunction<Simulation> fn;
        private double area;
        private double span;
        private final Map<Double, Double> histogram = new TreeMap<>();

        private Watch(ToDoubleFunction<Simulation> fn) {
            this.fn = fn;
        }

        private void add(Simulation s, double dt) {
            double v = fn.applyAsDouble(s);
            area += v * dt;
            span += dt;
            histogram.merge(v, dt, Double::sum);
        }

        public double mean() {
            return span > 0 ? area / span : Double.NaN;
        }

        public Map<Double, Double> distribution() {
            Map<Double, Double> d = new TreeMap<>();
            histogram.forEach((v, dt) -> d.put(v, dt / span));
            return d;
        }
    }
```

```
private final PetriSim sim;
private final PetriP[] places;
private final PetriT[] transitions;
private final Map<String, Integer> placeIndex = new HashMap<>();
private final Map<String, Integer> transitionIndex = new HashMap<>();
private final double warmup;
private final double[] markArea;
private final double[] bufferArea;
private final int[] entered;
private final int[] fired;
private final List<Watch> watches = new ArrayList<>();
private final List<Consumer<Simulation>> afterEvent = new ArrayList<>();
private final Map<PetriT, List<DoubleConsumer>> outActions = new HashMap<>();
private final List<String> inLog = new ArrayList<>();
private final List<String> outLog = new ArrayList<>();
private double time;
private long events;
private int traceEvents;
private Consumer<String> traceOut = s -> { };

public Simulation(PetriNet net, double warmup) {
    this.sim = new PetriSim(net);
    this.places = net.getListP();
    this.transitions = net.getListT();
    this.warmup = warmup;
    for (int i = 0; i < places.length; i++) {
        placeIndex.put(places[i].getName(), i);
    }
    for (int i = 0; i < transitions.length; i++) {
        transitionIndex.put(transitions[i].getName(), i);
    }
    markArea = new double[places.length];
    bufferArea = new double[transitions.length];
    entered = new int[transitions.length];
    fired = new int[transitions.length];
    sim.setFiringListener(this::onFiring);
}

private void onFiring(FiringPhase phase, PetriT t, double now) {
    int i = transitionIndex.get(t.getName());
    if (phase == FiringPhase.AFTER_ACT_IN) {
        if (now >= warmup) {
            entered[i]++;
        }
        if (events < traceEvents) {
            inLog.add(t.getName());
        }
    } else if (phase == FiringPhase.AFTER_ACT_OUT) {
        if (now >= warmup) {
            fired[i]++;
        }
        if (events < traceEvents) {
            outLog.add(t.getName());
        }
        for (DoubleConsumer action : outActions.getOrDefault(t, List.of())) {
            action.accept(now);
        }
    }
}

public void run(double simulationTime) {
    sim.setSimulationTime(simulationTime);
    sim.setTimeCurr(0);
    time = 0;
    sim.input();
    finishEvent();
    while (true) {
        double next = sim.getTimeMin();
        advance(Math.min(next, simulationTime), simulationTime);
        if (next > simulationTime) {
            break;
        }
        sim.setTimeCurr(next);
        sim.output();
        sim.input();
        events++;
        finishEvent();
```

```
    }
}

private void finishEvent() {
    for (Consumer<Simulation> hook : afterEvent) {
        hook.accept(this);
    }
    if (events < traceEvents) {
        traceOut.accept(state());
    }
    inLog.clear();
    outLog.clear();
}

/** Статистика за інтервал [time, to): маркування між подіями не змінюється. */
private void advance(double to, double simulationTime) {
    // середні значення бібліотеки – так само, як у PetriObjModel.go()
    if (to > 0) {
        double weight = to < simulationTime ? (to - time) / to : (simulationTime - time) / simulationTime;
        sim.doStatistics(weight);
    }
    double from = Math.max(time, warmup);
    if (to > from) {
        double dt = to - from;
        for (int i = 0; i < places.length; i++) {
            markArea[i] += places[i].getMark() * dt;
        }
        for (int i = 0; i < transitions.length; i++) {
            bufferArea[i] += transitions[i].getBuffer() * dt;
        }
        for (Watch w : watches) {
            w.add(this, dt);
        }
    }
    time = to;
}

private String state() {
    StringBuilder sb = new StringBuilder(String.format(Locale.ROOT, "t = %.4f%n", time));
    if (!outLog.isEmpty()) {
        sb.append("  вихід з переходів: ").append(grouped(outLog)).append('\n');
    }
    if (!inLog.isEmpty()) {
        sb.append("  вхід у переходи:   ").append(grouped(inLog)).append('\n');
    }
    List<String> marks = new ArrayList<>();
    for (PetriP p : places) {
        if (p.getMark() > 0) {
            marks.add(p.getName() + "=" + p.getMark());
        }
    }
    List<String> busy = new ArrayList<>();
    for (PetriT t : transitions) {
        if (t.getBuffer() > 0) {
            busy.add(String.format(Locale.ROOT, "%s=%d (вихід %.3f)", t.getName(), t.getBuffer(),
                    t.getMinTime()));
        }
    }
    sb.append("  маркування: ").append(String.join(", ", marks)).append('\n');
    sb.append("  у переходах: ").append(String.join(", ", busy)).append('\n');
    return sb.toString();
}

private static String grouped(List<String> names) {
    List<String> parts = new ArrayList<>();
    for (int i = 0; i < names.size(); ) {
        int j = i;
        while (j < names.size() && names.get(j).equals(names.get(i))) {
            j++;
        }
        parts.add(j - i == 1 ? names.get(i) : names.get(i) + " ×" + (j - i));
        i = j;
    }
    return String.join(", ", parts);
}

public void trace(int count, Consumer<String> out) {
```

```
        traceEvents = count;
        traceOut = out;
    }

    public void afterEvent(Consumer<Simulation> hook) {
        afterEvent.add(hook);
    }

    /** Дія, що виконується при кожному виході маркера з переходу (аргумент – момент виходу). */
    public void onOut(String transition, DoubleConsumer action) {
        outActions.computeIfAbsent(transitions[transitionIndex.get(transition)], k -> new ArrayList<>())
                .add(action);
    }

    public Watch watch(ToDoubleFunction<Simulation> fn) {
        Watch w = new Watch(fn);
        watches.add(w);
        return w;
    }

    public int mark(String place) {
        return places[placeIndex.get(place)].getMark();
    }

    public int buffer(String transition) {
        return transitions[transitionIndex.get(transition)].getBuffer();
    }

    public double meanMark(String place) {
        return markArea[placeIndex.get(place)] / period();
    }

    public double meanBuffer(String transition) {
        return bufferArea[transitionIndex.get(transition)] / period();
    }

    /** Середнє маркування, обчислене самою бібліотекою (PetriP.getMean, без розгону). */
    public double libraryMeanMark(String place) {
        return places[placeIndex.get(place)].getMean();
    }

    public double libraryMeanBuffer(String transition) {
        return transitions[transitionIndex.get(transition)].getMean();
    }

    public int fired(String transition) {
        return fired[transitionIndex.get(transition)];
    }

    public int entered(String transition) {
        return entered[transitionIndex.get(transition)];
    }

    public double time() {
        return time;
    }

    /** Тривалість збору статистики (без розгону). */
    public double period() {
        return time - warmup;
    }

    public long events() {
        return events;
    }

    public PetriP[] places() {
        return places;
    }

    public PetriT[] transitions() {
        return transitions;
    }
}
```

Файл Conveyor.java

```java
package lab6;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriT;

/** Завдання 1: конвеєрна система з п'ятьма обслуговуючими пристроями (задача 1 практикуму 5). */
public final class Conveyor {

    static final int DEVICES = 5;
    static final double ARRIVAL = 0.25;
    static final double SERVICE = 1.0;
    static final double STEP = 1.0;
    static final double BACK = 5.0;
    static final double TIME = 10000;
    static final double WARMUP = 1000;
    static final int RUNS = 20;

    private Conveyor() {
    }

    static NetBuilder net() {
        NetBuilder b = new NetBuilder("Conveyor");
        PetriP generator = b.place("Генератор", 1, 40, 420);
        PetriP[] part = new PetriP[DEVICES + 1];
        PetriP[] free = new PetriP[DEVICES + 1];
        for (int i = 1; i <= DEVICES; i++) {
            part[i] = b.place("Деталь біля П" + i, 0, x(i), 420);
            free[i] = b.place("П" + i + " вільний", 1, x(i) + 210, 270);
        }
        PetriP done = b.place("Оброблені деталі", 0, 800, 50);
        PetriT arrival = b.transition("Надходження", ARRIVAL, 150, 420);
        b.arc(generator, arrival);
        b.arc(arrival, generator);
        b.arc(arrival, part[1]);
        for (int i = 1; i <= DEVICES; i++) {
            PetriT service = b.expTransition("Обробка П" + i, SERVICE, x(i) + 100, 270);
            service.setPriority(1);
            b.arc(part[i], service);
            b.arc(free[i], service);
            b.arc(service, free[i]);
            b.arc(service, done);
            PetriT move = i < DEVICES
                    ? b.transition("Рух до П" + (i + 1), STEP, x(i) + 120, 420)
                    : b.transition("Повернення до П1", BACK, 760, 760);
            b.arc(part[i], move);
            b.arc(move, part[i < DEVICES ? i + 1 : 1]);
        }
        return b;
    }

    private static double x(int device) {
        return 280 + 240 * (device - 1);
    }

    /** Деталі в системі: біля пристроїв, на обробці та на конвеєрі (у переходах руху й повернення). */
    static int inSystem(Simulation s) {
        int n = 0;
        for (int i = 1; i <= DEVICES; i++) {
            n += s.mark("Деталь біля П" + i);
        }
        for (PetriT t : s.transitions()) {
            if (!t.getName().equals("Надходження")) {
                n += t.getBuffer();
            }
        }
        return n;
    }

    static int busyDevices(Simulation s) {
        int busy = 0;
        for (int i = 1; i <= DEVICES; i++) {
            busy += 1 - s.mark("П" + i + " вільний");
```

```
        }
        return busy;
    }

    /** Показники одного прогону. */
    record Run(double meanTime, double[] load, double[] share, double busy, Map<Double, Double> busyDist,
               double throughput, double returnsPerPart, double inSystem) {
    }

    static Run run(Simulation s) {
        Simulation.Watch busy = s.watch(Conveyor::busyDevices);
        s.run(TIME);
        double served = 0;
        double[] load = new double[DEVICES];
        double[] share = new double[DEVICES];
        for (int i = 1; i <= DEVICES; i++) {
            served += s.fired("Обробка П" + i);
            load[i - 1] = s.meanBuffer("Обробка П" + i);
        }
        double inSystem = 0;
        for (int i = 1; i <= DEVICES; i++) {
            share[i - 1] = s.fired("Обробка П" + i) / served;
            inSystem += s.meanMark("Деталь біля П" + i);
        }
        for (PetriT t : s.transitions()) {
            if (!t.getName().equals("Надходження")) {
                inSystem += s.meanBuffer(t.getName());
            }
        }
        double throughput = served / s.period();
        // середній час перебування за формулою Літтла: W = L / λ
        return new Run(inSystem / throughput, load, share, busy.mean(), busy.distribution(), throughput,
                s.fired("Повернення до П1") / served, inSystem);
    }

    static Map<String, Object> main(Output out) {
        out.println("ЗАВДАННЯ 1. Конвеєрна система (PetriSim)");
        out.printf("Надходження кожні %.2f хв, обробка exp(%.1f), рух між пристроями %.1f хв, "
                + "повернення %.1f хв, %d пристроїв%n%n", ARRIVAL, SERVICE, STEP, BACK, DEVICES);
        out.println("Трасування перших подій:\n");
        Simulation traced = new Simulation(net().build(), 0);
        traced.trace(12, out::println);
        traced.run(5);

        Simulation checked = new Simulation(net().build(), 0);
        int[] maxInSystem = {0};
        long[] checks = {0};
        checked.afterEvent(s -> {
            for (int i = 1; i <= DEVICES; i++) {
                Checks.check(s.mark("П" + i + " вільний") + s.buffer("Обробка П" + i) == 1, "пристрій П" + i);
            }
            Checks.check(s.mark("Генератор") + s.buffer("Надходження") == 1, "генератор");
            int created = s.fired("Надходження");
            Checks.check(created == s.mark("Оброблені деталі") + inSystem(s), "збереження деталей");
            maxInSystem[0] = Math.max(maxInSystem[0], inSystem(s));
            checks[0]++;
        });
        checked.run(TIME);
        double libraryDiff = Checks.libraryDifference(checked);
        out.printf("Перевірка інваріантів після кожної з %d подій: порушень немає; найбільше "
                + "деталей у системі %d%n",
                checks[0], maxInSystem[0]);
        out.printf("Середні значення бібліотеки (PetriP.getMean, PetriT.getMean) і власні статистики прогону "
                + "без розгону: найбільша розбіжність %.2e%n", libraryDiff);

        List<Run> runs = new ArrayList<>();
        for (int r = 0; r < RUNS; r++) {
            runs.add(run(new Simulation(net().build(), WARMUP)));
        }
        Map<String, Stats> table = new LinkedHashMap<>();
        table.put("середній час перебування, хв", stats(runs, Run::meanTime));
        for (int i = 0; i < DEVICES; i++) {
            int k = i;
            table.put("завантаження П" + (i + 1), stats(runs, r -> r.load()[k]));
        }
        table.put("середня кількість зайнятих пристроїв", stats(runs, Run::busy));
        table.put("пропускна здатність, деталей/хв", stats(runs, Run::throughput));
```

```
        table.put("повернень до П1 на одну деталь", stats(runs, Run::returnsPerPart));
        table.put("середня кількість деталей у системі", stats(runs, Run::inSystem));

        out.printf("%nРЕЗУЛЬТАТИ (%d прогонів по %.0f хв, розгін %.0f хв, 95 %% довірчий "
                + "інтервал):%n", RUNS, TIME,
                WARMUP);
        table.forEach((k, v) -> out.printf("  %-40s %s%n", k, v));
        Map<String, Double> dist = new LinkedHashMap<>();
        StringBuilder distText = new StringBuilder();
        for (int k = 0; k <= DEVICES; k++) {
            double key = k;
            double p = runs.stream().mapToDouble(r -> r.busyDist().getOrDefault(key, 0.0))
                    .average().orElse(0);
            dist.put(Integer.toString(k), p);
            distText.append(String.format(java.util.Locale.ROOT, "%sP(%d) = %.4f", k > 0 ? ", " : "", k, p));
        }
        out.println("Розподіл кількості зайнятих пристроїв: " + distText);
        double[] share = new double[DEVICES];
        StringBuilder shareText = new StringBuilder();
        for (int i = 0; i < DEVICES; i++) {
            int k = i;
            share[i] = runs.stream().mapToDouble(r -> r.share()[k]).average().orElse(0);
            shareText.append(String.format(java.util.Locale.ROOT, "%sП%d %.4f", i > 0 ? ", " : "", i + 1,
                    share[i]));
        }
        out.println("Частка деталей, оброблених на пристрої: " + shareText);
        double sumLoad = 0;
        for (int i = 1; i <= DEVICES; i++) {
            sumLoad += table.get("завантаження П" + i).mean();
        }
        double lambda = table.get("пропускна здатність, деталей/хв").mean();
        out.printf("%nПеревірка: пропускна здатність %.4f при інтенсивності надходження %.1f; "
                + "сума завантажень %.4f, λ·E[S] = %.1f%n", lambda, 1 / ARRIVAL, sumLoad, SERVICE / ARRIVAL);
        out.printf("Сума завантажень дорівнює середній кількості зайнятих пристроїв: %.4f і %.4f%n", sumLoad,
                table.get("середня кількість зайнятих пристроїв").mean());

        return Json.obj("table", table, "busy_distribution", dist, "share", share, "check_events", checks[0],
                "max_in_system", maxInSystem[0], "library_difference", libraryDiff, "runs", RUNS,
                "mean_times", runs.stream().map(Run::meanTime).toList());
    }

    static Stats stats(List<Run> runs, java.util.function.ToDoubleFunction<Run> f) {
        return Stats.of(runs.stream().map(r -> f.applyAsDouble(r)).toList());
    }
}
```

Файл Inventory.java

```
package lab6;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.function.ToDoubleFunction;

import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriT;

/** Завдання 2: управління запасами холодильників з періодичним переглядом (задача 4 практикуму 5). */
public final class Inventory {

    static final double DEMAND = 0.2;
    static final double LOST = 0.8;
    static final double BACKORDER = 0.2;
    static final double REVIEW = 4.0;
    static final double LEAD = 3.0;
    static final int LEVEL = 72;
    static final int REORDER = 18;
    static final double TIME = 5000;
    static final double WARMUP = 52;
    static final int RUNS = 100;

    private Inventory() {
    }

    static NetBuilder net() {
```

```
    NetBuilder b = new NetBuilder("Inventory");
    PetriP generator = b.place("Генератор попиту", 1, -60, 240);
    PetriP customer = b.place("Покупець", 0, 280, 240);
    PetriP stock = b.place("Запас", LEVEL, 640, 240);
    PetriP sold = b.place("Продано", 0, 1000, 400);
    PetriP backlog = b.place("Відкладений попит", 0, 600, 400);
    PetriP lost = b.place("Втрачені продажі", 0, 600, 100);
    PetriP shortage = b.place("Нестача до рівня", 0, 220, 560);
    PetriP timer = b.place("Таймер перегляду", 1, -60, 840);
    PetriP review = b.place("Перегляд", 0, 280, 840);
    PetriP order = b.place("Формування замовлення", 0, 640, 840);

    PetriT arrival = b.expTransition("Надходження покупця", DEMAND, 120, 240);
    b.arc(generator, arrival);
    b.arc(arrival, generator);
    b.arc(arrival, customer);
    PetriT sale = b.transition("Продаж", 0, 430, 240);
    sale.setPriority(3);
    b.arc(customer, sale);
    b.arc(stock, sale);
    b.arc(sale, sold);
    b.arc(sale, shortage);
    PetriT leave = b.transition("Відмова від покупки", 0, 430, 100);
    leave.setPriority(1);
    leave.setProbability(LOST);
    b.arc(customer, leave);
    b.arc(leave, lost);
    PetriT wait = b.transition("Повторне замовлення", 0, 430, 400);
    wait.setPriority(1);
    wait.setProbability(BACKORDER);
    b.arc(customer, wait);
    b.arc(wait, backlog);
    b.arc(wait, shortage);
    PetriT issue = b.transition("Видача відкладеного", 0, 780, 400);
    issue.setPriority(4);
    b.arc(stock, issue);
    b.arc(backlog, issue);
    b.arc(issue, sold);

    PetriT tick = b.transition("Перегляд запасу", REVIEW, 120, 840);
    b.arc(timer, tick);
    b.arc(tick, timer);
    b.arc(tick, review);
    // поточний стан запасу ≤ 18  ⇔  нестача до контрольного рівня 72 ≥ 54
    PetriT decide = b.transition("Рішення замовити", 0, 430, 840);
    decide.setPriority(2);
    b.arc(review, decide);
    b.info(shortage, decide, LEVEL - REORDER);
    b.arc(decide, order);
    PetriT skip = b.transition("Без замовлення", 0, 430, 980);
    b.arc(review, skip);
    // кожен холодильник нестачі – окремий маркер; усі вони входять у перехід в один момент
    // і одночасно виходять через 3 тижні, тобто утворюють одну партію
    PetriT delivery = b.transition("Доставка", LEAD, 820, 620);
    delivery.setPriority(1);
    b.arc(shortage, delivery);
    b.info(order, delivery, 1);
    b.arc(delivery, stock);
    PetriT close = b.transition("Кінець замовлення", 0, 860, 840);
    b.arc(order, close);
    return b;
}

/** Поточний стан запасу: наявний запас + замовлені раніше – невдоволений попит. */
static int position(Simulation s) {
    return s.mark("Запас") + s.buffer("Доставка") - s.mark("Відкладений попит");
}

record Run(double stock, double lostInterval, double lostRate, double lostShare, double backorderShare,
           double backorders, double stockout, double position, double orderSize, double orderInterval,
           double demandRate, double soldRate, List<Double> gaps) {
}

static Run run(Simulation s) {
    List<Double> lost = new ArrayList<>();
    int[] orders = {0};
    s.onOut("Відмова від покупки", t -> {
```

```
            if (t >= WARMUP) {
                lost.add(t);
            }
        });
        s.onOut("Рішення замовити", t -> {
            if (t >= WARMUP) {
                orders[0]++;
            }
        });
        Simulation.Watch empty = s.watch(x -> x.mark("Запас") == 0 ? 1 : 0);
        Simulation.Watch position = s.watch(Inventory::position);
        s.run(TIME);
        List<Double> gaps = new ArrayList<>();
        for (int i = 1; i < lost.size(); i++) {
            gaps.add(lost.get(i) - lost.get(i - 1));
        }
        double period = s.period();
        double demand = s.fired("Надходження покупця");
        double meanGap = gaps.stream().mapToDouble(Double::doubleValue).average().orElse(0);
        return new Run(s.meanMark("Запас"), meanGap,
                lost.size() / period, lost.size() / demand, s.fired("Повторне замовлення") / demand,
                s.meanMark("Відкладений попит"), empty.mean(), position.mean(),
                (double) s.entered("Доставка") / Math.max(orders[0], 1), period / orders[0], demand / period,
                (s.fired("Продаж") + s.fired("Видача відкладеного")) / period, gaps);
    }

    static Map<String, Object> main(Output out) {
        out.println("ЗАВДАННЯ 2. Управління запасами (PetriSim)");
        out.printf("Попит exp(%.1f) тижня, перегляд кожні %.0f тижні, рівень %d, точка замовлення "
                + "%d, доставка %.0f тижні, "
                + "втрата продажу з ймовірністю %.1f%n%n", DEMAND, REVIEW, LEVEL, REORDER, LEAD, LOST);
        out.println("Трасування подій навколо першого замовлення:\n");
        List<String> lines = new ArrayList<>();
        Simulation traced = new Simulation(net().build(), 0);
        traced.trace(Integer.MAX_VALUE, lines::add);
        traced.run(30);
        int first = indexOf(lines, "вхід у переходи:   Рішення замовити", 0);
        int delivery = indexOf(lines, "вихід з переходів: Доставка", first);
        List<String> excerpt = new ArrayList<>(lines.subList(first - 1, first + 3));
        excerpt.add("...\n");
        excerpt.addAll(lines.subList(delivery - 1, delivery + 2));
        excerpt.forEach(out::println);

        Simulation checked = new Simulation(net().build(), 0);
        long[] checks = {0};
        checked.afterEvent(s -> {
            Checks.check(position(s) + s.buffer("Продаж") + s.mark("Нестача до рівня") == LEVEL,
                    "стан запасу + нестача = " + LEVEL);
            checks[0]++;
        });
        checked.run(TIME);
        double libraryDiff = Checks.libraryDifference(checked);
        out.printf("Перевірка інваріанта «стан запасу + нестача до рівня = %d» після кожної з %d "
                + "подій: порушень немає%n", LEVEL, checks[0]);
        out.printf("Середні значення бібліотеки і власні статистики прогону без розгону: найбільша "
                + "розбіжність %.2e%n", libraryDiff);

        List<Run> runs = new ArrayList<>();
        for (int r = 0; r < RUNS; r++) {
            runs.add(run(new Simulation(net().build(), WARMUP)));
        }
        Map<String, Stats> table = new LinkedHashMap<>();
        table.put("середня кількість холодильників у запасі", stats(runs, Run::stock));
        table.put("середній час між втраченими продажами, тижні", stats(runs, Run::lostInterval));
        table.put("втрачених продажів за тиждень", stats(runs, Run::lostRate));
        table.put("частка втрачених продажів", stats(runs, Run::lostShare));
        table.put("частка відкладеного попиту", stats(runs, Run::backorderShare));
        table.put("середній невдоволений попит", stats(runs, Run::backorders));
        table.put("частка часу без запасу", stats(runs, Run::stockout));
        table.put("середній поточний стан запасу", stats(runs, Run::position));
        table.put("середній розмір замовлення", stats(runs, Run::orderSize));
        table.put("середній інтервал між замовленнями, тижні", stats(runs, Run::orderInterval));
        out.printf("%nРЕЗУЛЬТАТИ (%d прогонів по %.0f тижнів, розгін %.0f тижнів, 95 %% довірчий "
                + "інтервал):%n", RUNS,
                TIME, WARMUP);
        table.forEach((k, v) -> out.printf("  %-46s %s%n", k, v));
```

```
        List<Double> gaps = runs.stream().flatMap(r -> r.gaps().stream()).sorted().toList();
        double mean = gaps.stream().mapToDouble(Double::doubleValue).average().orElse(0);
        double ss = gaps.stream().mapToDouble(g -> (g - mean) * (g - mean)).sum();
        double sd = Math.sqrt(ss / (gaps.size() - 1));
        double median = gaps.get(gaps.size() / 2);
        double shortShare = gaps.stream().filter(g -> g < REVIEW).count() / (double) gaps.size();
        out.printf("%nІнтервали між втраченими продажами: медіана %.4f, коефіцієнт варіації %.4f, "
                + "частка інтервалів до 4 тижнів %.4f%n", median, sd / mean, shortShare);
        double demand = stats(runs, Run::demandRate).mean();
        double sold = stats(runs, Run::soldRate).mean();
        double lost = table.get("втрачених продажів за тиждень").mean();
        out.printf("%nПеревірка: інтенсивність попиту %.4f (задано %.0f), продано %.4f + втрачено "
                + "%.4f = %.4f%n",
                demand, 1 / DEMAND, sold, lost, sold + lost);
        out.printf("1 / інтенсивність втрачених продажів = %.4f тижня%n", 1 / lost);
        return Json.obj("table", table, "gaps", Json.obj("median", median, "cv", sd / mean, "short_share",
                shortShare), "demand_rate", demand, "sold_rate", sold, "check_events", checks[0],
                "library_difference", libraryDiff, "runs", RUNS, "trace", String.join("\n", excerpt));
    }

    private static int indexOf(List<String> lines, String text, int from) {
        for (int i = from; i < lines.size(); i++) {
            if (lines.get(i).contains(text)) {
                return i;
            }
        }
        throw new IllegalStateException(text);
    }

    static Stats stats(List<Run> runs, ToDoubleFunction<Run> f) {
        return Stats.of(runs.stream().map(f::applyAsDouble).toList());
    }
}
```

Файл Servers.java

```
package lab6;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.function.ToDoubleFunction;

import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriT;

/**
 * Завдання 3: багатопотокова обробка запитів на головному (C1) і допоміжному (C2) серверах зі
 * спільною базою даних. Одиниця часу – мілісекунда.
 */
public final class Servers {

    /** Параметри моделі; rate – інтенсивність надходження запитів, запитів/мс. */
    record Config(double rate, double pAux, int queue, int[] threads, int[] cores, double[] cpu,
                  int connections, double db, int routingPriority) {

        static Config base() {
            return new Config(0.2, 0.3, 1000, new int[] {32, 16}, new int[] {8, 4}, new double[] {40, 50}, 10,
                    20, 3);
        }

        Config withRate(double r) {
            return new Config(r, pAux, queue, threads, cores, cpu, connections, db, routingPriority);
        }

        Config withThreads(int main, int aux) {
            return new Config(rate, pAux, queue, new int[] {main, aux}, cores, cpu, connections, db,
                    routingPriority);
        }

        Config withRoutingPriority(int priority) {
            return new Config(rate, pAux, queue, threads, cores, cpu, connections, db, priority);
        }

        double serverRate(int s) {
            return rate * (s == 0 ? 1 - pAux : pAux);
```

```
    }
}

static final double TIME = 600_000;
static final double WARMUP = 20_000;
static final int RUNS = 20;
static final double EXP_TIME = 300_000;
static final int EXP_RUNS = 10;

private Servers() {
}

static String sfx(int s) {
    return " C" + (s + 1);
}

static NetBuilder net(Config c) {
    NetBuilder b = new NetBuilder("Servers");
    PetriP generator = b.place("Генератор", 1, 40, 500);
    PetriP request = b.place("Запит", 0, 270, 500);
    PetriP connections = b.place("З'єднання БД", c.connections(), 1100, 500);
    PetriT arrival = b.expTransition("Надходження", 1 / c.rate(), 150, 500);
    b.arc(generator, arrival);
    b.arc(arrival, generator);
    b.arc(arrival, request);
    for (int s = 0; s < 2; s++) {
        double y = s == 0 ? 300 : 700;
        double ry = s == 0 ? 150 : 920; // ряд позицій-ресурсів сервера
        String n = sfx(s);
        PetriT route = b.transition("На" + n, 0, 380, y);
        route.setPriority(c.routingPriority());
        route.setProbability(s == 0 ? 1 - c.pAux() : c.pAux());
        PetriP incoming = b.place("Запит" + n, 0, 500, y);
        PetriP slots = b.place("Місця" + n, c.queue(), s == 0 ? 700 : 530, ry);
        PetriP queue = b.place("Черга" + n, 0, 740, y);
        PetriP threads = b.place("Потоки" + n, c.threads()[s], s == 0 ? 920 : 770, ry);
        PetriP ready = b.place("Готові" + n, 0, 980, y);
        PetriP cores = b.place("Ядра" + n, c.cores()[s], s == 0 ? 1160 : 1000, ry);
        PetriP computed = b.place("Обчислено" + n, 0, 1220, y);
        PetriP responses = b.place("Відповіді" + n, 0, 1460, y);
        PetriP rejected = b.place("Відхилено" + n, 0, 740, s == 0 ? 420 : 580);

        b.arc(request, route);
        b.arc(route, incoming);
        PetriT accept = b.transition("Прийом" + n, 0, 620, y);
        accept.setPriority(2);
        b.arc(incoming, accept);
        b.arc(slots, accept);
        b.arc(accept, queue);
        PetriT reject = b.transition("Відмова" + n, 0, 620, s == 0 ? 420 : 580);
        reject.setPriority(1);
        b.arc(incoming, reject);
        b.arc(reject, rejected);
        PetriT take = b.transition("Потік" + n, 0, 860, y);
        b.arc(queue, take);
        b.arc(threads, take);
        b.arc(take, ready);
        b.arc(take, slots);
        PetriT compute = b.expTransition("Обробка" + n, c.cpu()[s], 1100, y);
        b.arc(ready, compute);
        b.arc(cores, compute);
        b.arc(compute, cores);
        b.arc(compute, computed);
        PetriT write = b.expTransition("Запис" + n, c.db(), 1340, y);
        b.arc(computed, write);
        b.arc(connections, write);
        b.arc(write, connections);
        b.arc(write, threads);
        b.arc(write, responses);
    }
    return b;
}

/** Запити, що перебувають на сервері s: у черзі, у потоках, на обчисленні та записі в БД. */
static String[] stagePlaces(int s) {
    return new String[] {"Черга" + sfx(s), "Готові" + sfx(s), "Обчислено" + sfx(s)};
}
```

```
record ServerRun(double lambda, double response, double queueWait, double coreWait, double compute,
                 double dbWait, double dbWrite, double rejectedShare, double coreLoad, double threadsBusy,
                 double queueMean, int queueMax) {
}

record Run(ServerRun[] server, double response, double auxShare, double dbBusy, double rejectedShare) {
}

static Run run(Config c, double time, double warmup) {
    Simulation s = new Simulation(net(c).build(), warmup);
    int[] queueMax = new int[2];
    s.afterEvent(x -> {
        for (int k = 0; k < 2; k++) {
            queueMax[k] = Math.max(queueMax[k], x.mark("Черга" + sfx(k)));
        }
    });
    s.run(time);
    ServerRun[] servers = new ServerRun[2];
    double inSystem = 0;
    double done = 0;
    double dbBusy = 0;
    double rejected = 0;
    for (int k = 0; k < 2; k++) {
        String n = sfx(k);
        double lambda = s.fired("Запис" + n) / s.period();
        double[] parts = {s.meanMark("Черга" + n), s.meanMark("Готові" + n), s.meanBuffer("Обробка" + n),
                s.meanMark("Обчислено" + n), s.meanBuffer("Запис" + n)};
        double total = 0;
        for (double v : parts) {
            total += v;
        }
        // формула Літтла для кожного етапу: W = L / λ
        servers[k] = new ServerRun(lambda, total / lambda, parts[0] / lambda, parts[1] / lambda,
                parts[2] / lambda, parts[3] / lambda, parts[4] / lambda,
                s.fired("Відмова" + n) / (double) s.fired("На" + n),
                s.meanBuffer("Обробка" + n) / c.cores()[k], c.threads()[k] - s.meanMark("Потоки" + n),
                s.meanMark("Черга" + n), queueMax[k]);
        inSystem += total;
        done += lambda;
        dbBusy += parts[4];
        rejected += s.fired("Відмова" + n);
    }
    double arrived = s.fired("Надходження");
    double routed = s.fired("На C1") + s.fired("На C2");
    return new Run(servers, inSystem / done, s.fired("На C2") / routed,
            dbBusy / c.connections(), rejected / arrived);
}

/** Час відповіді за мережею Джексона (потоки й черга необмежені): M/M/K для ядер, M/M/M для БД. */
static double[] analytic(Config c) {
    double db = Queueing.sojournMMc(c.connections(), c.rate(), c.db());
    double[] w = new double[3];
    for (int k = 0; k < 2; k++) {
        w[k] = Queueing.sojournMMc(c.cores()[k], c.serverRate(k), c.cpu()[k]) + db;
    }
    w[2] = (1 - c.pAux()) * w[0] + c.pAux() * w[1];
    return w;
}

static List<Run> runs(Config c, int count, double time, double warmup) {
    List<Run> list = new ArrayList<>();
    for (int i = 0; i < count; i++) {
        list.add(run(c, time, warmup));
    }
    return list;
}

static Stats stats(List<Run> runs, ToDoubleFunction<Run> f) {
    return Stats.of(runs.stream().map(f::applyAsDouble).toList());
}

static Map<String, Object> main(Output out) {
    Config c = Config.base();
    out.println("ЗАВДАННЯ 3. Обробка запитів на двох серверах зі спільною БД (PetriSim), час у мс");
    out.printf("Запити exp(%.0f) мс (%.0f запитів/с), на допоміжний сервер з ймовірністю %.1f, "
            + "черга до %d%n",
```

```
        1 / c.rate(), c.rate() * 1000, c.pAux(), c.queue());
for (int k = 0; k < 2; k++) {
    out.printf("Сервер%s: потоків %d, ядер %d, обчислення exp(%.0f) мс%n", sfx(k), c.threads()[k],
            c.cores()[k], c.cpu()[k]);
}
out.printf("БД: з'єднань %d, запис exp(%.0f) мс%n%n", c.connections(), c.db());

out.println("Трасування перших подій:\n");
Simulation traced = new Simulation(net(c).build(), 0);
traced.trace(10, out::println);
traced.run(30);

Simulation checked = new Simulation(net(c).build(), 0);
long[] checks = {0};
checked.afterEvent(s -> {
    int inSystem = s.mark("Запит");
    int finished = 0;
    for (int k = 0; k < 2; k++) {
        String n = sfx(k);
        Checks.check(s.mark("Потоки" + n) + s.buffer("Потік" + n) + s.mark("Готові" + n)
                + s.buffer("Обробка" + n) + s.mark("Обчислено" + n) + s.buffer("Запис" + n)
                == c.threads()[k], "потоки" + n);
        Checks.check(s.mark("Ядра" + n) + s.buffer("Обробка" + n) == c.cores()[k], "ядра" + n);
        Checks.check(s.mark("Місця" + n) + s.buffer("Прийом" + n) + s.mark("Черга" + n)
                + s.buffer("Потік" + n) + n) == c.queue(), "черга" + n);
        inSystem += s.mark("Запит" + n) + s.mark("Черга" + n) + s.mark("Готові" + n)
                + s.mark("Обчислено" + n);
        finished += s.mark("Відповіді" + n) + s.mark("Відхилено" + n);
    }
    for (PetriT t : s.transitions()) {
        if (!t.getName().equals("Надходження")) {
            inSystem += t.getBuffer();
        }
    }
    int db = s.mark("З'єднання БД") + s.buffer("Запис С1") + s.buffer("Запис С2");
    Checks.check(db == c.connections(),
            "з'єднання БД");
    Checks.check(s.fired("Надходження") == inSystem + finished, "збереження запитів");
    checks[0]++;
});
checked.run(TIME);
double libraryDiff = Checks.libraryDifference(checked);
out.printf("Перевірка інваріантів після кожної з %d подій: порушень немає%n", checks[0]);
out.printf("Середні значення бібліотеки і власні статистики прогону без розгону: найбільша "
        + "розбіжність %.2e%n", libraryDiff);

List<Run> base = runs(c, RUNS, TIME, WARMUP);
Map<String, Stats> table = new LinkedHashMap<>();
table.put("середній час відповіді (усі запити), мс", stats(base, Run::response));
for (int k = 0; k < 2; k++) {
    int i = k;
    String n = " (С" + (k + 1) + ")";
    table.put("час відповіді" + n + ", мс", stats(base, r -> r.server()[i].response()));
    table.put("  очікування в черзі" + n + ", мс", stats(base, r -> r.server()[i].queueWait()));
    table.put("  очікування ядра" + n + ", мс", stats(base, r -> r.server()[i].coreWait()));
    table.put("  обчислення" + n + ", мс", stats(base, r -> r.server()[i].compute()));
    table.put("  очікування з'єднання БД" + n + ", мс", stats(base, r -> r.server()[i].dbWait()));
    table.put("  запис у БД" + n + ", мс", stats(base, r -> r.server()[i].dbWrite()));
    table.put("пропускна здатність" + n + ", запитів/с",
            stats(base, r -> r.server()[i].lambda() * 1000));
    table.put("завантаження ядер" + n, stats(base, r -> r.server()[i].coreLoad()));
    table.put("зайнято потоків" + n, stats(base, r -> r.server()[i].threadsBusy()));
    table.put("середня довжина черги" + n, stats(base, r -> r.server()[i].queueMean()));
}
table.put("завантаження з'єднань БД", stats(base, Run::dbBusy));
table.put("частка запитів на С2", stats(base, Run::auxShare));
table.put("частка відхилених запитів", stats(base, Run::rejectedShare));
out.printf("%nРЕЗУЛЬТАТИ (%d прогонів по %.0f с, розгін %.0f с, 95 %% довірчий інтервал):%n", RUNS,
        TIME / 1000, WARMUP / 1000);
table.forEach((k, v) -> out.printf("  %-46s %s%n", k, v));

double[] w = analytic(c);
out.printf("%nПеревірка за мережею Джексона (M/M/K для ядер, M/M/%d для БД): час відповіді "
        + "С1 %.4f, С2 %.4f, усі %.4f мс%n", c.connections(), w[0], w[1], w[2]);
for (int k = 0; k < 2; k++) {
    out.printf("Завантаження ядер С%d: λ·E[S]/K = %.4f%n", k + 1,
            c.serverRate(k) * c.cpu()[k] / c.cores()[k]);
```

```
        }
        out.printf("Завантаження з'єднань БД: λ·E[S]/M = %.4f%n", c.rate() * c.db() / c.connections());
        double lam = (table.get("пропускна здатність (C1), запитів/с").mean()
                + table.get("пропускна здатність (C2), запитів/с").mean());
        out.printf("Сумарна пропускна здатність %.4f запитів/с при інтенсивності надходження %.1f%n", lam,
                c.rate() * 1000);

        List<Run> wrong = runs(c.withRoutingPriority(0), 5, EXP_TIME, WARMUP);
        Stats wrongShare = stats(wrong, Run::auxShare);
        out.printf("%nПереходи розподілу з пріоритетом 0 (як у переході «Надходження»): частка "
                + "запитів на C2 %s замість %.1f%n", wrongShare, c.pAux());

        out.printf("%nЕКСПЕРИМЕНТ 1: кількість потоків головного сервера (%d прогонів по %.0f с)%n", EXP_RUNS,
                EXP_TIME / 1000);
        out.println("  потоків C1   час відповіді C1, мс    очікування в черзі, мс   завантаження ядер  "
                + "зайнято потоків   відхилено");
        List<Object> threadRows = new ArrayList<>();
        for (int n : new int[] {8, 10, 12, 14, 16, 24, 32, 64}) {
            List<Run> r = runs(c.withThreads(n, c.threads()[1]), EXP_RUNS, EXP_TIME, WARMUP);
            Stats resp = stats(r, x -> x.server()[0].response());
            Stats wait = stats(r, x -> x.server()[0].queueWait());
            Stats load = stats(r, x -> x.server()[0].coreLoad());
            Stats busy = stats(r, x -> x.server()[0].threadsBusy());
            Stats rej = stats(r, x -> x.server()[0].rejectedShare());
            out.printf("  %9d   %22s  %22s  %17s  %15s  %10.4f%n", n, resp.format(2), wait.format(2),
                    load.format(4), busy.format(2), rej.mean());
            threadRows.add(Json.obj("threads", n, "response", resp, "queue_wait", wait, "core_load", load,
                    "threads_busy", busy, "rejected", rej));
        }

        out.printf("%nЕКСПЕРИМЕНТ 2: інтенсивність надходження (%d прогонів по %.0f с)%n", EXP_RUNS,
                EXP_TIME / 1000);
        out.println("  запитів/с  потоки C1/C2  час відповіді, мс        теорія, мс   відхилено   "
                + "черга C1 max  черга C2 max");
        List<Object> loadRows = new ArrayList<>();
        for (double perSecond : new double[] {100, 150, 200, 230, 250, 280, 400}) {
            for (int[] threads : new int[][] {{32, 16}, {12, 6}}) {
                Config cfg = c.withRate(perSecond / 1000).withThreads(threads[0], threads[1]);
                List<Run> r = runs(cfg, EXP_RUNS, EXP_TIME, WARMUP);
                Stats resp = stats(r, Run::response);
                Stats rej = stats(r, Run::rejectedShare);
                int q1 = r.stream().mapToInt(x -> x.server()[0].queueMax()).max().orElse(0);
                int q2 = r.stream().mapToInt(x -> x.server()[1].queueMax()).max().orElse(0);
                double theory = analytic(cfg)[2];
                out.printf("  %9.0f  %12s  %22s  %10.2f   %9.4f   %12d  %12d%n", perSecond,
                        threads[0] + "/" + threads[1], resp.format(2), theory, rej.mean(), q1, q2);
                loadRows.add(Json.obj("rate", perSecond, "threads", List.of(threads[0], threads[1]),
                        "response", resp, "response1", stats(r, x -> x.server()[0].response()),
                        "response2", stats(r, x -> x.server()[1].response()), "theory", theory,
                        "rejected", rej, "queue_max", List.of(q1, q2)));
            }
        }
        return Json.obj("config", Json.obj("rate_per_s", c.rate() * 1000, "p_aux", c.pAux(),
                        "queue", c.queue(),
                        "threads", List.of(c.threads()[0], c.threads()[1]),
                        "cores", List.of(c.cores()[0], c.cores()[1]), "cpu", List.of(c.cpu()[0], c.cpu()[1]),
                        "connections", c.connections(), "db", c.db()),
                "table", table, "analytic", List.of(w[0], w[1], w[2]), "check_events", checks[0],
                "library_difference", libraryDiff, "wrong_priority_share", wrongShare, "threads", threadRows,
                "load", loadRows, "runs", RUNS);
    }
}
```

Файл NetExport.java

```
package lab6;

import java.awt.Color;
import java.awt.Font;
import java.awt.Graphics2D;
import java.awt.Rectangle;
import java.awt.RenderingHints;
import java.awt.image.BufferedImage;
import java.io.File;
import java.io.FileOutputStream;
import java.io.ObjectOutputStream;
import java.nio.file.Files;
```

```
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.IdentityHashMap;
import java.util.List;
import java.util.Map;

import javax.imageio.ImageIO;

import ua.stetsenkoinna.graphnet.GraphArcIn;
import ua.stetsenkoinna.graphnet.GraphArcOut;
import ua.stetsenkoinna.graphnet.GraphCanvasModel;
import ua.stetsenkoinna.graphnet.GraphPetriNet;
import ua.stetsenkoinna.graphnet.GraphPetriObjModel;
import ua.stetsenkoinna.graphnet.GraphPetriPlace;
import ua.stetsenkoinna.graphnet.GraphPetriTransition;
import ua.stetsenkoinna.graphpresentation.FileUse;
import ua.stetsenkoinna.petriobj.ArcIn;
import ua.stetsenkoinna.petriobj.ArcOut;
import ua.stetsenkoinna.petriobj.PetriNet;
import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriT;
import ua.stetsenkoinna.pnml.PnmlModelGenerator;
import ua.stetsenkoinna.pnml.PnmlModelParser;
import ua.stetsenkoinna.theme.CanvasColor;
import ua.stetsenkoinna.theme.CanvasPalette;

/**
 * Збереження мережі Петрі засобами графічного редактора PetriObjModelPaint: графічна мережа
 * GraphPetriNet (як після «Save Graph net», файл .pns) та її зображення .png, документ .pnml
 * (як «Save As…») і java-метод (як «Save net as method»).
 */
public final class NetExport {

    private static final int MARGIN = 60;
    private static final int SCALE = 2;

    private NetExport() {
    }

    static void main(Path netsDir, Path results) throws Exception {
        export(Conveyor.net(), netsDir.resolve("conveyor"), "conveyor", results);
        export(Inventory.net(), netsDir.resolve("inventory"), "inventory", results);
        export(Servers.net(Servers.Config.base()), netsDir.resolve("servers"), "servers", results);
    }

    static void export(NetBuilder builder, Path dir, String file, Path results) throws Exception {
        Files.createDirectories(dir);
        PetriNet net = builder.build();
        GraphPetriNet graph = graphNet(builder, net);

        graph.createPetriNet(net.getName());
        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(dir.resolve(file + ".pns")
                .toFile()))) {
            oos.writeObject(graph);
        }
        ImageIO.write(paint(graph), "png", dir.resolve(file + ".png").toFile());
        File pnml = dir.resolve(file + ".pnml").toFile();
        new PnmlModelGenerator().generate(new GraphCanvasModel(net.getName(), graph).toObjModel(), pnml);
        Output.write(dir.resolve(file + "_method.txt"), new FileUse().saveNetAsMethod(graph));
        Output.write(results.resolve("net_" + file + ".json"), Json.write(describe(graph.getPetriNet())));

        GraphPetriObjModel parsed = new PnmlModelParser().parse(pnml);
        PetriNet back = GraphPetriObjModel.createPetriSim(parsed.getObject(0)).getNet();
        boolean same = signature(back).equals(signature(graph.getPetriNet()));
        System.out.printf("%s: %d позицій, %d переходів, %d вхідних і %d вихідних дуг; "
                + "мережа, прочитана з .pnml, %s з вихідною%n", dir.resolve(file), net.getListP().length,
                net.getListT().length, net.getArcIn().length, net.getArcOut().length, same ? "збігається" :
"НЕ збігається");
        if (!same) {
            throw new IllegalStateException("PNML " + pnml);
        }
    }

    /** Графічна мережа з координатами, заданими в NetBuilder (так само будує її редактор). */
    static GraphPetriNet graphNet(NetBuilder b, PetriNet net) {
        Map<Object, Object> graphOf = new IdentityHashMap<>();
        ArrayList<GraphPetriPlace> places = new ArrayList<>();
```

```
        ArrayList<GraphPetriTransition> transitions = new ArrayList<>();
        int id = 0;
        for (PetriP p : net.getListP()) {
            GraphPetriPlace g = new GraphPetriPlace(p, id++);
            g.setNewCoordinates(b.position(p));
            places.add(g);
            graphOf.put(p, g);
        }
        for (PetriT t : net.getListT()) {
            GraphPetriTransition g = new GraphPetriTransition(t, id++);
            g.setNewCoordinates(b.position(t));
            transitions.add(g);
            graphOf.put(t, g);
        }
        ArrayList<GraphArcIn> arcsIn = new ArrayList<>();
        for (ArcIn arc : net.getArcIn()) {
            GraphArcIn g = new GraphArcIn(arc);
            g.settingNewArc(places.get(arc.getNumP()));
            g.finishSettingNewArc(transitions.get(arc.getNumT()));
            g.setPetriElements();
            g.changeBorder();
            arcsIn.add(g);
        }
        ArrayList<GraphArcOut> arcsOut = new ArrayList<>();
        for (ArcOut arc : net.getArcOut()) {
            GraphArcOut g = new GraphArcOut(arc);
            g.settingNewArc(transitions.get(arc.getNumT()));
            g.finishSettingNewArc(places.get(arc.getNumP()));
            g.setPetriElements();
            g.changeBorder();
            arcsOut.add(g);
            // зустрічні дуги між тими самими елементами редактор розводить паралельно
            for (GraphArcIn in : arcsIn) {
                if (in.getBeginElement() == g.getEndElement() && in.getEndElement() == g.getBeginElement()) {
                    in.twoArcs(g);
                }
            }
        }
        return new GraphPetriNet(net, places, transitions, arcsIn, arcsOut);
    }

    /** Зображення мережі так, як її малює полотно редактора (GraphPetriNet.paintGraphPetriNet). */
    static BufferedImage paint(GraphPetriNet graph) {
        Rectangle box = null;
        for (GraphPetriPlace p : graph.getGraphPetriPlaceList()) {
            box = union(box, p.getBounds());
        }
        for (GraphPetriTransition t : graph.getGraphPetriTransitionList()) {
            box = union(box, t.getBounds());
        }
        int width = box.width + 2 * MARGIN + 80;
        int height = box.height + 2 * MARGIN + 40;
        BufferedImage image = new BufferedImage(width * SCALE, height * SCALE, BufferedImage.TYPE_INT_RGB);
        Graphics2D g = image.createGraphics();
        g.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
        g.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING, RenderingHints.VALUE_TEXT_ANTIALIAS_ON);
        Color background = CanvasPalette.current().get(CanvasColor.CANVAS_BACKGROUND);
        g.setColor(background);
        g.fillRect(0, 0, image.getWidth(), image.getHeight());
        g.scale(SCALE, SCALE);
        g.translate(MARGIN + 40 - box.x, MARGIN + 20 - box.y);
        g.setFont(new Font(Font.SANS_SERIF, Font.PLAIN, 12));
        graph.paintGraphPetriNet(g, g);
        g.dispose();
        return image;
    }

    private static Rectangle union(Rectangle box, Rectangle r) {
        return box == null ? new Rectangle(r) : box.union(r);
    }

    /** Опис структури мережі у форматі, яким описано мережі практикуму 5. */
    static Map<String, Object> describe(PetriNet net) {
        List<Object> places = new ArrayList<>();
        for (PetriP p : net.getListP()) {
            places.add(Json.obj("name", p.getName(), "mark", p.getMark()));
        }
```

```
        List<Object> transitions = new ArrayList<>();
        for (PetriT t : net.getListT()) {
            List<Object> in = new ArrayList<>();
            List<Object> info = new ArrayList<>();
            List<Object> out = new ArrayList<>();
            for (ArcIn a : net.getArcIn()) {
                if (a.getNumT() == t.getNumber()) {
                    (a.getIsInf() ? info : in).add(Json.obj("place", net.getListP()[a.getNumP()].getName(),
                            "n", a.getQuantity()));
                }
            }
            for (ArcOut a : net.getArcOut()) {
                if (a.getNumT() == t.getNumber()) {
                    out.add(Json.obj("place", net.getListP()[a.getNumP()].getName(), "n", a.getQuantity()));
                }
            }
            transitions.add(Json.obj("name", t.getName(),
                    "distribution", t.getDistribution() == null ? "const" : t.getDistribution(),
                    "mean", t.getParameter(), "priority", t.getPriority(), "probability", t.getProbability(),
                    "inputs", in, "info_inputs", info, "outputs", out));
        }
        return Json.obj("name", net.getName(), "places", places, "transitions", transitions);
    }

    /** Склад мережі без урахування порядку елементів: позиції, переходи та дуги як рядки. */
    static List<String> signature(PetriNet net) {
        List<String> items = new ArrayList<>();
        PetriP[] p = net.getListP();
        PetriT[] t = net.getListT();
        for (PetriP place : p) {
            items.add("P " + place.getName() + " " + place.getMark());
        }
        for (PetriT tr : t) {
            items.add("T " + tr.getName() + " " + tr.getParameter() + " " + tr.getDistribution() + " "
                    + tr.getPriority() + " " + tr.getProbability());
        }
        for (ArcIn a : net.getArcIn()) {
            items.add("I " + p[a.getNumP()].getName() + " " + t[a.getNumT()].getName() + " " + a.getQuantity()
                    + " " + a.getIsInf());
        }
        for (ArcOut a : net.getArcOut()) {
            items.add("O " + t[a.getNumT()].getName() + " " + p[a.getNumP()].getName() + " "
                    + a.getQuantity());
        }
        items.sort(null);
        return items;
    }
}
```

Файл Queueing.java

```
package lab6;

/** Формули теорії масового обслуговування для перевірки моделі серверів. */
final class Queueing {

    private Queueing() {
    }

    /** Ймовірність очікування в системі M/M/c (формула Ерланга C), a = λ/μ. */
    static double erlangC(int c, double a) {
        double term = 1;
        double sum = 1;
        for (int k = 1; k < c; k++) {
            term *= a / k;
            sum += term;
        }
        double last = term * a / c / (1 - a / c);
        return last / (sum + last);
    }

    /** Середній час перебування в системі M/M/c з інтенсивністю λ і середнім часом обслуговування s. */
    static double sojournMMc(int c, double lambda, double s) {
        double a = lambda * s;
        if (a >= c) {
            return Double.POSITIVE_INFINITY;
        }
```

```
        return erlangC(c, a) * s / (c - a) + s;
    }
}
```

Файл Checks.java

```
package lab6;

import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriT;

/** Допоміжні перевірки моделей. */
final class Checks {

    private Checks() {
    }

    static void check(boolean ok, String what) {
        if (!ok) {
            throw new IllegalStateException("порушено інваріант: " + what);
        }
    }

    /**
     * Найбільша розбіжність між середніми, які обчислює бібліотека (PetriP.getMean, PetriT.getMean),
     * і власними статистиками Simulation; має сенс для прогону без розгону.
     */
    static double libraryDifference(Simulation s) {
        double max = 0;
        for (PetriP p : s.places()) {
            max = Math.max(max, Math.abs(s.meanMark(p.getName()) - s.libraryMeanMark(p.getName())));
        }
        for (PetriT t : s.transitions()) {
            max = Math.max(max, Math.abs(s.meanBuffer(t.getName()) - s.libraryMeanBuffer(t.getName())));
        }
        return max;
    }
}
```

Файл Stats.java

```
package lab6;

import java.util.Collection;
import java.util.Locale;

/** Середнє, стандартне відхилення та півширина 95 % довірчого інтервалу за t-розподілом Стьюдента. */
public record Stats(double mean, double ci, int n) {

    public static Stats of(Collection<Double> values) {
        int n = values.size();
        double mean = values.stream().mapToDouble(Double::doubleValue).sum() / n;
        if (n < 2) {
            return new Stats(mean, 0.0, n);
        }
        double ss = values.stream().mapToDouble(v -> (v - mean) * (v - mean)).sum();
        return new Stats(mean, tQuantile975(n - 1) * Math.sqrt(ss / (n - 1) / n), n);
    }

    /** Квантиль рівня 0,975 t-розподілу (розклад Корніша – Фішера, похибка < 1e-4 при ν ≥ 5). */
    static double tQuantile975(int nu) {
        double z = 1.959963984540054;
        double z3 = z * z * z;
        double z5 = z3 * z * z;
        double z7 = z5 * z * z;
        return z + (z3 + z) / (4.0 * nu) + (5 * z5 + 16 * z3 + 3 * z) / (96.0 * nu * nu)
                + (3 * z7 + 19 * z5 + 17 * z3 - 15 * z) / (384.0 * nu * nu * nu);
    }

    public String format(int digits) {
        return String.format(Locale.ROOT, "%." + digits + "f ± %." + digits + "f", mean, ci);
    }

    @Override
    public String toString() {
        return format(4);
```

```
    }
}
```

Файл Json.java

```
package lab6;

import java.util.Collection;
import java.util.Iterator;
import java.util.LinkedHashMap;
import java.util.Locale;
import java.util.Map;

/** Запис результатів у JSON (Map, Collection, Number, String, Boolean, Stats). */
public final class Json {

    private Json() {
    }

    /** Впорядкований словник з пар «ключ, значення». */
    public static Map<String, Object> obj(Object... keyValues) {
        Map<String, Object> m = new LinkedHashMap<>();
        for (int i = 0; i < keyValues.length; i += 2) {
            m.put((String) keyValues[i], keyValues[i + 1]);
        }
        return m;
    }

    public static String write(Object value) {
        StringBuilder sb = new StringBuilder();
        append(sb, value, 0);
        return sb.append('\n').toString();
    }

    private static void append(StringBuilder sb, Object v, int indent) {
        switch (v) {
            case null -> sb.append("null");
            case Stats s -> append(sb, obj("mean", s.mean(), "ci", s.ci()), indent);
            case Map<?, ?> m -> container(sb, '{', '}', m.entrySet().iterator(), indent, true);
            case Collection<?> c -> container(sb, '[', ']', c.iterator(), indent, false);
            case double[] a -> append(sb, java.util.Arrays.stream(a).boxed().toList(), indent);
            case Double d -> sb.append(Double.isFinite(d) ? String.format(Locale.ROOT, "%.10g", d) : "null");
            case Number n -> sb.append(n);
            case Boolean b -> sb.append(b);
            default -> sb.append('"').append(v.toString().replace("\\", "\\\\").replace("\"", "\\\"")
                    .replace("\n", "\\n")).append('"');
        }
    }

    private static void container(StringBuilder sb, char open, char close, Iterator<?> it, int indent,
                                  boolean map) {
        sb.append(open);
        boolean first = true;
        while (it.hasNext()) {
            Object item = it.next();
            sb.append(first ? "\n" : ",\n").append(" ".repeat(indent + 1));
            first = false;
            if (map) {
                Map.Entry<?, ?> e = (Map.Entry<?, ?>) item;
                append(sb, String.valueOf(e.getKey()), indent + 1);
                sb.append(": ");
                append(sb, e.getValue(), indent + 1);
            } else {
                append(sb, item, indent + 1);
            }
        }
        if (!first) {
            sb.append('\n').append(" ".repeat(indent));
        }
        sb.append(close);
    }
}
```

Файл Output.java

```java
package lab6;

import java.io.IOException;
import java.io.UncheckedIOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Locale;

/** Вивід програми: одночасно на консоль і у файл результатів. */
public final class Output {

    private final StringBuilder text = new StringBuilder();

    public void println(String line) {
        System.out.println(line);
        text.append(line).append('\n');
    }

    public void println() {
        println("");
    }

    public void printf(String format, Object... args) {
        String s = String.format(Locale.ROOT, format, args);
        System.out.print(s);
        text.append(s);
    }

    public void save(Path file) {
        write(file, text.toString());
    }

    public static void write(Path file, String content) {
        try {
            Files.createDirectories(file.getParent());
            Files.writeString(file, content);
        } catch (IOException e) {
            throw new UncheckedIOException(e);
        }
    }
}
```

Файл Main.java

```java
package lab6;

import java.nio.file.Path;
import java.util.List;
import java.util.Map;
import java.util.function.Function;

/** Запуск моделей: java lab6.Main [conveyor] [inventory] [servers] [export]. */
public final class Main {

    private static final Path RESULTS = Path.of("results");

    private Main() {
    }

    public static void main(String[] args) throws Exception {
        java.util.logging.LogManager.getLogManager().reset(); // журнал бібліотеки не потрібен
        List<String> tasks = args.length > 0 ? List.of(args)
                : List.of("conveyor", "inventory", "servers", "export");
        for (String task : tasks) {
            switch (task) {
                case "conveyor" -> task("task1", Conveyor::main);
                case "inventory" -> task("task2", Inventory::main);
                case "servers" -> task("task3", Servers::main);
                case "export" -> NetExport.main(Path.of("nets"), RESULTS);



                default -> throw new IllegalArgumentException(task);
            }
        }
    }
```

```
    private static void task(String name, Function<Output, Map<String, Object>> model) {
        Output out = new Output();
        long start = System.nanoTime();
        Map<String, Object> json = model.apply(out);
        out.save(RESULTS.resolve(name + "_output.txt"));
        Output.write(RESULTS.resolve(name + ".json"), Json.write(json));
        System.out.printf("[%s: %.1f c]%n%n", name, (System.nanoTime() - start) / 1e9);
    }
}
```

Файл compare_lab5.py

```
"""Порівняння мереж і результатів практикуму 6 (PetriSim) з мережами та результатами практикуму 5."""
import json
import math
import sys
from pathlib import Path

from scipy import stats

HERE = Path(__file__).resolve().parent
LAB5 = HERE.parent / "lab5" / "results"
RESULTS = HERE / "results"
PAIRS = [("conveyor", "net1_conveyor", "task1", "task1"), ("inventory", "net4_inventory", "task4", "task2")]


def load(path):
    return json.loads(path.read_text(encoding="utf-8"))


def structure(net):
    """Склад мережі без урахування порядку елементів; перенесення маркерів (carry) у PetriSim не існує."""
    items = {("P", p["name"], p["mark"]) for p in net["places"]}
    for t in net["transitions"]:
        items.add(("T", t["name"], t["distribution"], round(t["mean"], 9), t["priority"],
                   round(t["probability"], 9)))
        items |= {("I", a["place"], t["name"], a["n"]) for a in t["inputs"]}
        items |= {("II", a["place"], t["name"], a["n"]) for a in t["info_inputs"]}
        items |= {("O", t["name"], a["place"], a["n"]) for a in t["outputs"]}
    return items


def compare_results(lab5, lab6, runs5, runs6):
    """Середні з 95 % довірчими інтервалами двох серій прогонів і p-значення критерію Велча."""
    old = {r["metric"]: r for r in lab5["table"]}
    rows = []
    for metric, new in lab6["table"].items():
        if metric not in old:
            continue
        m5, c5, m6, c6 = old[metric]["mean"], old[metric]["ci"], new["mean"], new["ci"]
        s5 = c5 / stats.t.ppf(0.975, runs5 - 1)
        s6 = c6 / stats.t.ppf(0.975, runs6 - 1)
        se = math.hypot(s5, s6)
        z = (m6 - m5) / se if se > 0 else 0.0
        df = se ** 4 / (s5 ** 4 / (runs5 - 1) + s6 ** 4 / (runs6 - 1)) if se > 0 else 1
        p = 2 * stats.t.sf(abs(z), df)
        rows.append({"metric": metric, "lab5": [m5, c5], "lab6": [m6, c6], "diff": m6 - m5, "p": p})
    return rows


def main():
    report, ok = {}, True
    for name, net5, res5, res6 in PAIRS:
        s5 = structure(load(LAB5 / f"{net5}.json"))
        s6 = structure(load(RESULTS / f"net_{name}.json"))
        same = s5 == s6
        ok &= same
        print(f"{name}: структура мережі {'збігається' if same else 'НЕ збігається'} з мережею практикуму 5 "
              f"({sum(1 for i in s6 if i[0] == 'P')} позицій, {sum(1 for i in s6 if i[0] == 'T')} переходів, "
              f"{sum(1 for i in s6 if i[0] in ('I', 'II', 'O'))} дуг)")
        for item in sorted(s5 ^ s6, key=str):
            print("   ", "лише ЛР5" if item in s5 else "лише ЛР6", item)
        lab6 = load(RESULTS / f"{res6}.json")
        runs5 = 20 if name == "conveyor" else 100
        rows = compare_results(load(LAB5 / f"{res5}.json"), lab6, runs5, lab6["runs"])
        for r in rows:
            (m5, c5), (m6, c6) = r["lab5"], r["lab6"]
```

```
            print(f"  {r['metric']:46} ЛР5 {m5:9.4f} ± {c5:.4f}   ЛР6 {m6:9.4f} ± {c6:.4f}   p =
{r['p']:.3f}")
        report[name] = {"same_structure": same, "rows": rows}
    (RESULTS / "lab5_comparison.json").write_text(json.dumps(report, ensure_ascii=False, indent=1),
                                                  encoding="utf-8")
    sys.exit(0 if ok else 1)


if __name__ == "__main__":
    main()
```

3. Результати роботи програми:

Завдання 1:

Бібліотека PetriObjModelPaint описує мережу Петрі об'єктом `PetriNet` зі списків позицій `PetriP` (ім'я, початкове маркування), переходів `PetriT` (затримка – стала або випадкова з розподілом exp, unif, norm; пріоритет; ймовірність спрацювання), вхідних дуг `ArcIn` (кратність, ознака інформаційної дуги) і вихідних дуг `ArcOut`. Переходи багатоканальні: кожен маркер, що увійшов у перехід, отримує свій момент виходу в масиві `timeOut`, кількість маркерів у переході – `buffer`. Клас `PetriSim` реалізує алгоритм імітації: метод `input()` поки можливо формує список переходів, для яких виконано умову запуску (`findActiveT`), вибирає з нього перехід (`doConflikt`: найвищий пріоритет, серед переходів з однаковим пріоритетом – випадково за ймовірностями) і виконує вхід маркерів (`actIn`); метод `eventMin()` знаходить найближчий момент виходу; метод `output()` виконує вихід маркерів з усіх переходів, момент виходу яких дорівнює поточному часу (`actOut`).

Клас `Simulation` викликає ці методи в тому самому порядку, що й метод `go` класу `PetriObjModel` для одного Петрі-об'єкта (саме так мережу запускає графічний редактор): `input()`, потім у циклі – просування часу до `getTimeMin()`, `output()`, `input()`. Власний метод `go()` класу `PetriSim` використати не вдалося з двох причин. По-перше, він зупиняє імітацію, щойно після виходу маркерів усі переходи порожні (метод `isStop()` перевіряє лише буфери переходів), тому мережа з генератором, який повертає маркер у вхідну позицію, зупиняється після першої ж події. По-друге, у методі `step()` середні значення оновлюються з вагою $\Delta t / T_{мод}$, тобто як експоненційно зважене, а не звичайне середнє за часом: для тестової СМО M/M/1 із завантаженням 0,8 воно дало 0,50 замість 0,80. Тому статистика накопичується в `Simulation` (інтеграли маркування позицій і кількості маркерів у переходах з урахуванням періоду розгону, якого бібліотека не підтримує), а паралельно викликається `doStatistics` з вагою $(t_{n+1} - t_n) / t_{n+1}$, як у `PetriObjModel.go`. Для прогону без розгону середні значення бібліотеки

(`PetriP.getMean`, `PetriT.getMean`) і власні збігаються з точністю до 10^{-10}, що перевіряє обидва способи. Входи й виходи переходів фіксуються спостерігачем `TransitionFiringListener`, який бібліотека викликає до і після `actIn` / `actOut`.

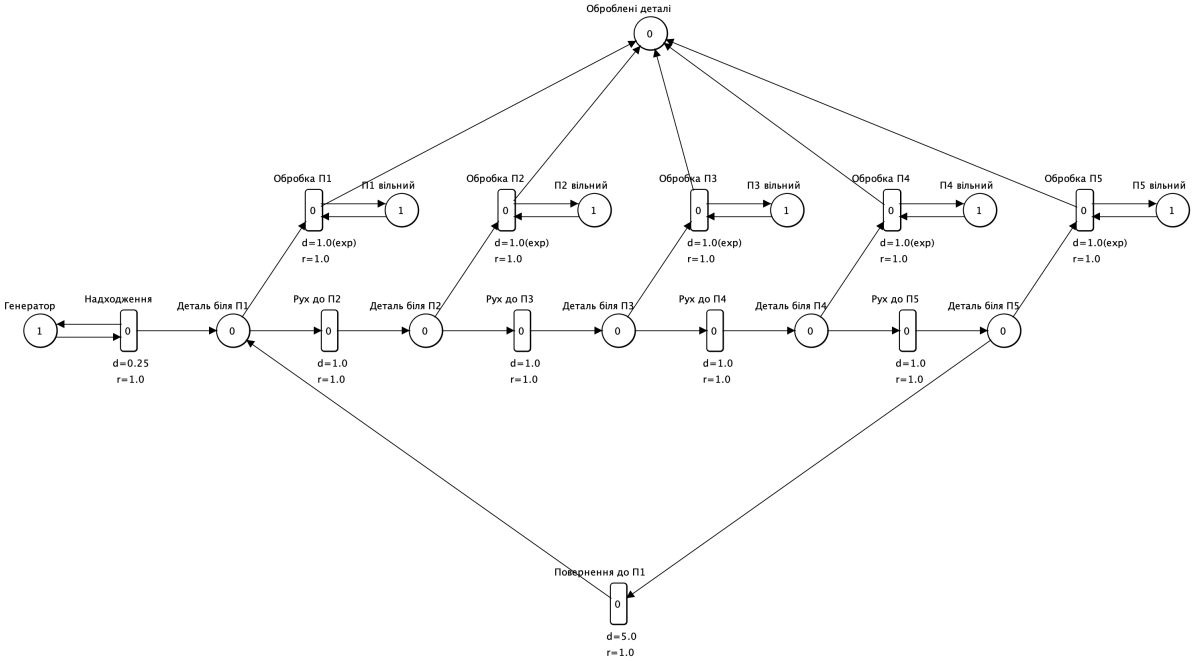


Рисунок 1 – Мережа Петрі конвеєрної системи (PetriObjModelPaint)

Таблиця 1 – Переходи мережі конвеєрної системи

Перехід	Вхідні позиції	Вихідні позиції	Затримка	Пріоритет	Ймовірність
Надходження	Генератор	Генератор, Деталь біля П1	0,25	0	1,0
Обробка П1	Деталь біля П1, П1 вільний	П1 вільний, Оброблені деталі	exp(1,0)	1	1,0
Рух до П2	Деталь біля П1	Деталь біля П2	1	0	1,0
Обробка П2	Деталь біля П2, П2 вільний	П2 вільний, Оброблені деталі	exp(1,0)	1	1,0
Рух до П3	Деталь біля П2	Деталь біля П3	1	0	1,0
Обробка П3	Деталь біля П3, П3 вільний	П3 вільний, Оброблені деталі	exp(1,0)	1	1,0
Рух до П4	Деталь біля П3	Деталь біля П4	1	0	1,0
Обробка П4	Деталь біля П4, П4 вільний	П4 вільний, Оброблені деталі	exp(1,0)	1	1,0
Рух до П5	Деталь біля П4	Деталь біля П5	1	0	1,0
Обробка П5	Деталь біля П5, П5 вільний	П5 вільний, Оброблені деталі	exp(1,0)	1	1,0
Повернення до П1	Деталь біля П5	Деталь біля П1	5	0	1,0

Мережа (рис. 1, табл. 1) повністю повторює мережу практикуму 5: 12 позицій, 11 переходів, 33 дуги, ті самі затримки й пріоритети. Початкове маркування: Генератор – 1, П1 вільний – 1, П2 вільний – 1, П3 вільний – 1, П4 вільний – 1, П5 вільний – 1. Перехід «Надходження» з затримкою 0,25 хв і петлею через позицію «Генератор» породжує деталь кожні 0,25 хв. Деталь біля пристрою Пі займає його, якщо в позиції «Пі вільний» є маркер (перехід «Обробка Пі», exp(1), пріоритет 1), інакше з меншим пріоритетом 0 рухається конвеєром до наступного пристрою за 1 хв, а після П5 повертається до П1 за 5 хв. Усі переходи мають необмежену кількість каналів – у PetriSim це властивість будь-якого переходу, кількість одночасних обробок обмежує ресурсна позиція «Пі вільний».

Програмна перевірка `compare_lab5.py` порівнює склад мереж практикумів 5 і 6 (позиції з маркуванням, переходи з параметрами, дуги з кратностями) і підтверджує, що вони збігаються. Єдина відмінність – маркери в PetriSim неіменовані, тож перенесення конкретної деталі з позиції в позицію, яким у практикумі 5 вимірювався час перебування кожної деталі, тут неможливе. Середній час перебування обчислюється за формулою Літтла $W = L / \lambda$, де L – середня кількість деталей у системі (позиції «Деталь біля Пі» і маркери в переходах обробки, руху та повернення), λ – пропускна здатність. Для однієї траєкторії $\int N(t)dt$ дорівнює сумі часів перебування деталей, тому оцінка відрізняється від прямого вимірювання лише крайовими ефектами. Трасування перших подій:

```
t = 0.0000
  вхід у переходи:   Надходження
  маркування: П1 вільний=1, П2 вільний=1, П3 вільний=1, П4 вільний=1, П5 вільний=1
  у переходах: Надходження=1 (вихід 0.250)

t = 0.2500
  вихід з переходів: Надходження
  вхід у переходи:   Обробка П1, Надходження
  маркування: П2 вільний=1, П3 вільний=1, П4 вільний=1, П5 вільний=1
  у переходах: Надходження=1 (вихід 0.500), Обробка П1=1 (вихід 1.549)

t = 0.5000
  вихід з переходів: Надходження
  вхід у переходи:   Надходження, Рух до П2
  маркування: П2 вільний=1, П3 вільний=1, П4 вільний=1, П5 вільний=1
  у переходах: Надходження=1 (вихід 0.750), Обробка П1=1 (вихід 1.549), Рух до П2=1 (вихід 1.500)

t = 0.7500
  вихід з переходів: Надходження
  вхід у переходи:   Надходження, Рух до П2
  маркування: П2 вільний=1, П3 вільний=1, П4 вільний=1, П5 вільний=1
  у переходах: Надходження=1 (вихід 1.000), Обробка П1=1 (вихід 1.549), Рух до П2=2 (вихід 1.500)

t = 1.0000
  вихід з переходів: Надходження
  вхід у переходи:   Надходження, Рух до П2
  маркування: П2 вільний=1, П3 вільний=1, П4 вільний=1, П5 вільний=1
  у переходах: Надходження=1 (вихід 1.250), Обробка П1=1 (вихід 1.549), Рух до П2=3 (вихід 1.500)

t = 1.2500
  вихід з переходів: Надходження
  вхід у переходи:   Надходження, Рух до П2
  маркування: П2 вільний=1, П3 вільний=1, П4 вільний=1, П5 вільний=1
  у переходах: Надходження=1 (вихід 1.500), Обробка П1=1 (вихід 1.549), Рух до П2=4 (вихід 1.500)

t = 1.5000
  вихід з переходів: Надходження, Рух до П2
  вхід у переходи:   Обробка П2, Рух до П2, Надходження
  маркування: П3 вільний=1, П4 вільний=1, П5 вільний=1
  у переходах: Надходження=1 (вихід 1.750), Обробка П1=1 (вихід 1.549), Рух до П2=4 (вихід
1.750), Обробка П2=1 (вихід 1.670)
```

Трасування відповідає задачі: о 0,25 хв перша деталь займає П1, наступні деталі (0,5; 0,75; …) застають П1 зайнятим і входять у перехід «Рух до П2» (у ньому одночасно кілька маркерів з різними моментами виходу); через 1 хв кожна з них з'являється біля П2 і займає його, якщо він вільний. Вихід маркера з переходу «Надходження» і вхід у нього відбуваються в один момент, тож генератор працює без пропусків.

```
Перевірка інваріантів після кожної з 79958 подій: порушень немає; найбільше деталей у системі 70
Середні значення бібліотеки (PetriP.getMean, PetriT.getMean) і власні статистики прогону без
розгону: найбільша розбіжність 8.00e-11

РЕЗУЛЬТАТИ (20 прогонів по 10000 хв, розгін 1000 хв, 95 % довірчий інтервал):
  середній час перебування, хв             9.7139 ± 0.2042
  завантаження П1                          0.8850 ± 0.0007
  завантаження П2                          0.8553 ± 0.0008
  завантаження П3                          0.8149 ± 0.0016
  завантаження П4                          0.7600 ± 0.0028
  завантаження П5                          0.6919 ± 0.0041
  середня кількість зайнятих пристроїв     4.0071 ± 0.0084
  пропускна здатність, деталей/хв          4.0004 ± 0.0010
  повернень до П1 на одну деталь           0.7592 ± 0.0224
  середня кількість деталей у системі      38.8595 ± 0.8176
Розподіл кількості зайнятих пристроїв: P(0) = 0.0005, P(1) = 0.0081, P(2) = 0.0554, P(3) =
0.1995, P(4) = 0.3930, P(5) = 0.3435
Частка деталей, оброблених на пристрої: П1 0.2211, П2 0.2134, П3 0.2029, П4 0.1896, П5 0.1730

Перевірка: пропускна здатність 4.0004 при інтенсивності надходження 4.0; сума завантажень 4.0071,
λ·E[S] = 4.0
Сума завантажень дорівнює середній кількості зайнятих пристроїв: 4.0071 і 4.0071
```

Після кожної з 79958 подій контрольного прогону перевірялися P-інваріанти мережі: «Пі вільний» + маркери в «Обробка Пі» = 1 для кожного пристрою, «Генератор» + маркер у «Надходження» = 1, а також збереження деталей: кількість створених деталей дорівнює сумі оброблених і тих, що перебувають у системі. Порушень немає. Сума завантажень пристроїв дорівнює середній кількості зайнятих пристроїв і $\lambda \cdot E[S] = 4 \cdot 1 = 4$, пропускна здатність збігається з інтенсивністю надходження 4 деталі/хв: усі деталі рано чи пізно обробляються.

Таблиця 2 – Конвеєрна система: практикум 5 і PetriSim

Показник	**ЛР5 (Python)**	**ЛР6 (PetriSim)**	**p**
середній час перебування, хв	9,6205 ± 0,2386	9,7139 ± 0,2042	0,54
завантаження П1	0,8851 ± 0,0007	0,8850 ± 0,0007	0,76
завантаження П2	0,8543 ± 0,0010	0,8553 ± 0,0008	0,10
завантаження П3	0,8132 ± 0,0017	0,8149 ± 0,0016	0,15
завантаження П4	0,7582 ± 0,0025	0,7600 ± 0,0028	0,31
завантаження П5	0,6883 ± 0,0035	0,6919 ± 0,0041	0,17
середня кількість зайнятих пристроїв	3,9991 ± 0,0082	4,0071 ± 0,0084	0,16
пропускна здатність, деталей/хв	3,9998 ± 0,0007	4,0004 ± 0,0010	0,32
повернень до П1 на одну деталь	0,7488 ± 0,0257	0,7592 ± 0,0224	0,53
середня кількість деталей у системі	38,4688 ± 0,9470	38,8595 ± 0,8176	0,52

Результати PetriSim і програми практикуму 5 (табл. 2, однакові умови: 20 прогонів по 10000 хв, розгін 1000 хв) статистично не відрізняються: для всіх показників довірчі інтервали перетинаються, p-значення критерію Велча більші за 0,05. Оскільки модель практикуму 5 була незалежно перевірена, це підтверджує правильність реалізації мережі в PetriSim.

Середній час перебування деталі в системі – 9,71 хв, хоча середній час обробки лише 1 хв: деталь у середньому 0,76 разу повертається до першого пристрою (5 хв) і проходить кілька відрізків конвеєра. Завантаження пристроїв спадає вздовж конвеєра – від 0,885 для П1 до 0,692 для П5, бо до наступних пристроїв доходять лише деталі, які застали попередні зайнятими. У середньому зайнято 4,01 пристрої з 5; усі п'ять зайняті 34,4 % часу, чотири – 39,3 %. Система перевантажена майже до межі ($\lambda \cdot E[S] = 4$ при 5 пристроях), і через відсутність місць перед пристроями в системі постійно перебуває близько 39 деталей, більшість з яких рухається конвеєром.

Завдання 2:

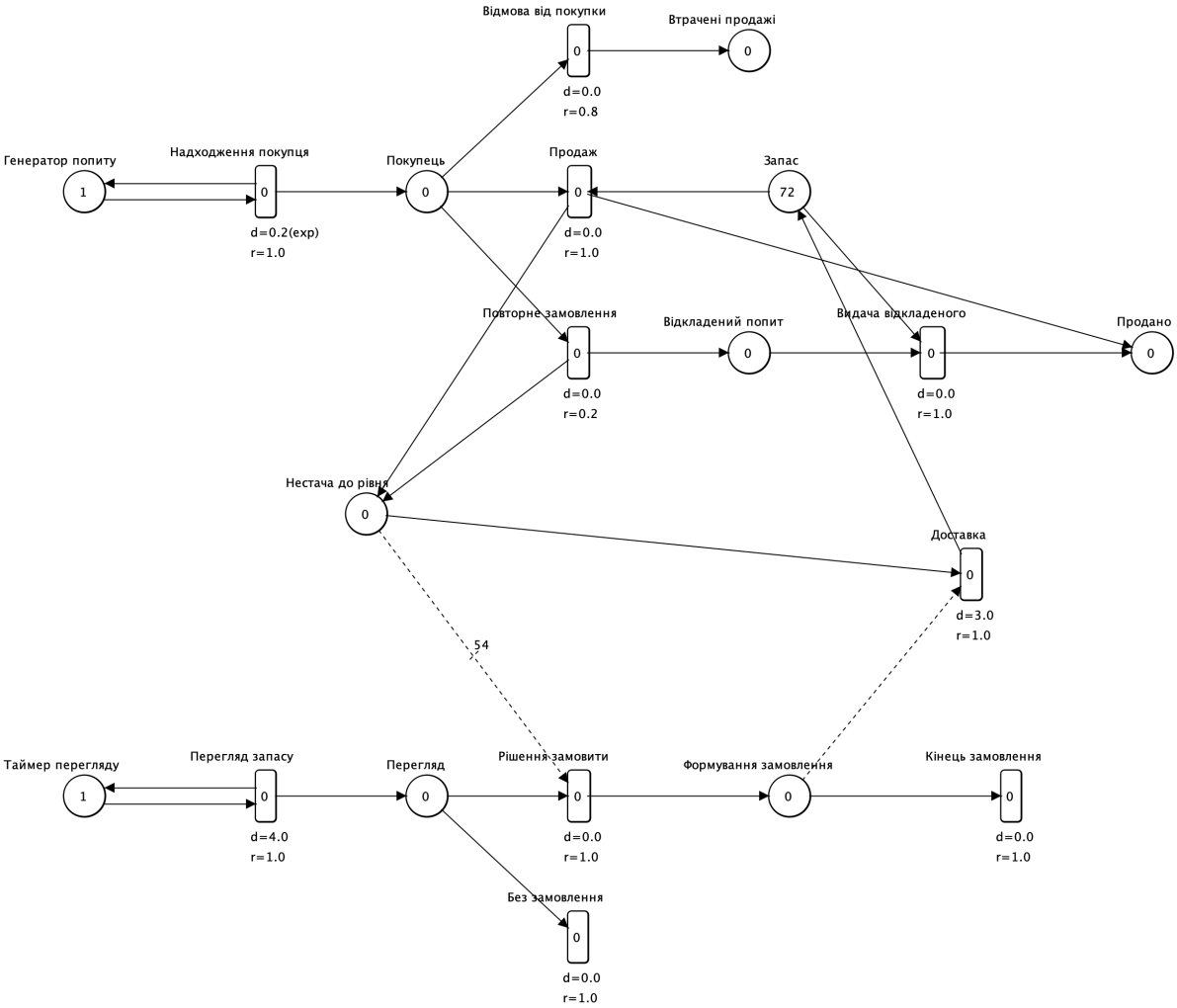


Рисунок 2 – Мережа Петрі системи управління запасами

Таблиця 3 – Переходи мережі управління запасами

Перехід	Вхідні позиції	Вихідні позиції	Затримка	Пріоритет	Ймовірність
Надходження покупця	Генератор попиту	Генератор попиту, Покупець	exp(0,2)	0	1,0
Продаж	Покупець, Запас	Продано, Нестача до рівня	0	3	1,0
Відмова від покупки	Покупець	Втрачені продажі	0	1	0,8
Повторне замовлення	Покупець	Відкладений попит, Нестача до рівня	0	1	0,2
Видача відкладеного	Запас, Відкладений попит	Продано	0	4	1,0
Перегляд запасу	Таймер перегляду	Таймер перегляду, Перегляд	4	0	1,0
Рішення замовити	Перегляд, Нестача до рівня ×54 (інф.)	Формування замовлення	0	2	1,0
Без замовлення	Перегляд	—	0	0	1,0
Доставка	Нестача до рівня, Формування замовлення ×1 (інф.)	Запас	3	1	1,0
Кінець замовлення	Формування замовлення	—	0	0	1,0

Мережа (рис. 2, табл. 3) збігається з мережею практикуму 5 (перевірено програмою `compare_lab5.py`). Початкове маркування: Генератор попиту – 1, Запас – 72, Таймер перегляду – 1. Покупець (exp(0,2) тижня) з найвищим серед своїх альтернатив пріоритетом 3 купує холодильник, якщо він є в запасі; інакше з пріоритетом 1 і ймовірностями 0,8 / 0,2 відмовляється або робить повторне замовлення. Позиція «Нестача до рівня» містить різницю між контрольним рівнем 72 і поточним станом запасу, тому умова «поточний стан ≤ 18» перевіряється інформаційною дугою кратності 54 до переходу «Рішення замовити» (пріоритет 2 вищий, ніж у «Без замовлення»). Під час замовлення інформаційна дуга з позиції «Формування замовлення» дозволяє переходу «Доставка» (затримка 3 тижні) прийняти всі маркери нестачі в той самий момент, тому вони одночасно виходять у «Запас» однією партією; «Кінець замовлення» з нижчим пріоритетом закриває замовлення. Відкладений попит обслуговується першим (пріоритет 4).

Ця мережа використовує все, що відрізняє PetriSim від найпростішої мережі: інформаційні дуги, дугу з кратністю 54, переходи-стоки без вихідних дуг, переходи з нульовою затримкою, конфлікт за ймовірностями. Перехід з нульовою затримкою в PetriSim спрацьовує як окрема подія в той самий момент часу: маркер

входить у перехід під час `input()`, а виходить під час наступного `output()`. Трасування навколо першого замовлення:

```
t = 15.9979
  вихід з переходів: Продаж
  маркування: Запас=1, Продано=71, Нестача до рівня=71
  у переходах: Надходження покупця=1 (вихід 16.084), Перегляд запасу=1 (вихід 16.000)

t = 16.0000
  вихід з переходів: Перегляд запасу
  вхід у переходи:   Рішення замовити, Перегляд запасу
  маркування: Запас=1, Продано=71, Нестача до рівня=71
  у переходах: Надходження покупця=1 (вихід 16.084), Перегляд запасу=1 (вихід 20.000), Рішення
замовити=1 (вихід 16.000)

t = 16.0000
  вихід з переходів: Рішення замовити
  вхід у переходи:   Доставка ×71, Кінець замовлення
  маркування: Запас=1, Продано=71
  у переходах: Надходження покупця=1 (вихід 16.084), Перегляд запасу=1 (вихід 20.000),
Доставка=71 (вихід 19.000), Кінець замовлення=1 (вихід 16.000)

t = 16.0000
  вихід з переходів: Кінець замовлення
  маркування: Запас=1, Продано=71
  у переходах: Надходження покупця=1 (вихід 16.084), Перегляд запасу=1 (вихід 20.000),
Доставка=71 (вихід 19.000)

...

t = 18.6417
  вихід з переходів: Відмова від покупки
  маркування: Продано=72, Відкладений попит=2, Втрачені продажі=10, Нестача до рівня=3
  у переходах: Надходження покупця=1 (вихід 19.353), Перегляд запасу=1 (вихід 20.000),
Доставка=71 (вихід 19.000)

t = 19.0000
  вихід з переходів: Доставка ×71
  вхід у переходи:   Видача відкладеного ×2
  маркування: Запас=69, Продано=72, Втрачені продажі=10, Нестача до рівня=3
  у переходах: Надходження покупця=1 (вихід 19.353), Видача відкладеного=2 (вихід 19.000),
Перегляд запасу=1 (вихід 20.000)

t = 19.0000
  вихід з переходів: Видача відкладеного ×2
  маркування: Запас=69, Продано=74, Втрачені продажі=10, Нестача до рівня=3
  у переходах: Надходження покупця=1 (вихід 19.353), Перегляд запасу=1 (вихід 20.000)
```

У момент t = 16 тижнів перегляд застає нестачу ≥ 54, спрацьовує «Рішення замовити», і в той самий момент усі маркери нестачі входять у «Доставка» (замовлення доводить стан запасу до 72); через 3 тижні вони разом виходять у «Запас», після чого першим обслуговується відкладений попит.

```
Перевірка інваріанта «стан запасу + нестача до рівня = 72» після кожної з 53964 подій: порушень немає
Середні значення бібліотеки і власні статистики прогону без розгону: найбільша розбіжність 1.46e-11

РЕЗУЛЬТАТИ (100 прогонів по 5000 тижнів, розгін 52 тижнів, 95 % довірчий інтервал):
  середня кількість холодильників у запасі       28.8333 ± 0.0255
  середній час між втраченими продажами, тижні   2.3285 ± 0.0169
  втрачених продажів за тиждень                  0.4286 ± 0.0032
  частка втрачених продажів                      0.0857 ± 0.0006
  частка відкладеного попиту                     0.0214 ± 0.0002
  середній невдоволений попит                    0.1357 ± 0.0018
  частка часу без запасу                         0.1070 ± 0.0007
  середній поточний стан запасу                  42.4144 ± 0.0214
  середній розмір замовлення                     63.4159 ± 0.0544
  середній інтервал між замовленнями, тижні      13.8568 ± 0.0252

Інтервали між втраченими продажами: медіана 0.1848, коефіцієнт варіації 2.6800, частка інтервалів до 4 тижнів 0.8726

Перевірка: інтенсивність попиту 5.0005 (задано 5), продано 4.5718 + втрачено 0.4286 = 5.0005
1 / інтенсивність втрачених продажів = 2.3330 тижня
```

Інваріант «стан запасу + нестача до рівня = 72» (з урахуванням маркера всередині переходу «Продаж») виконувався після кожної з 53964 подій. Балансова перевірка: продано + втрачено = 5,000 холодильника за тиждень при інтенсивності попиту 5,000 (задано 5).

Таблиця 4 – Управління запасами: практикум 5 і PetriSim

Показник	ЛР5 (Python)	ЛР6 (PetriSim)	p
середня кількість холодильників у запасі	28,8412 ± 0,0242	28,8333 ± 0,0255	0,66
середній час між втраченими продажами, тижні	2,3400 ± 0,0231	2,3285 ± 0,0169	0,43
втрачених продажів за тиждень	0,4270 ± 0,0042	0,4286 ± 0,0032	0,54
частка втрачених продажів	0,0854 ± 0,0008	0,0857 ± 0,0006	0,56
частка відкладеного попиту	0,0213 ± 0,0003	0,0214 ± 0,0002	0,57
середній невдоволений попит	0,1348 ± 0,0024	0,1357 ± 0,0018	0,55
частка часу без запасу	0,1065 ± 0,0009	0,1070 ± 0,0007	0,43
середній поточний стан запасу	42,4201 ± 0,0275	42,4144 ± 0,0214	0,75
середній розмір замовлення	63,3888 ± 0,0778	63,4159 ± 0,0544	0,57
середній інтервал між замовленнями, тижні	13,8542 ± 0,0300	13,8568 ± 0,0252	0,89

Результати PetriSim (100 прогонів по 5000 тижнів, розгін 52 тижні) збігаються з результатами практикуму 5 (табл. 4) у межах довірчих інтервалів, усі p-значення більші за 0,05. Середня кількість холодильників у запасі – 28,83, середній час між продажами, що не відбулися, – 2,33 тижня. Втрачені продажі групуються: медіана інтервалу лише 0,18 тижня, коефіцієнт варіації 2,68. Запас закінчується наприкінці циклу замовлення (10,7 % часу), і тоді втрати йдуть щільною серією; решту часу їх немає. Замовлення розміщується в середньому раз на 13,9 тижня обсягом 63,4 холодильника, тож частка втрачених продажів 8,6 %

зумовлена поєднанням рідкого перегляду (4 тижні) і доставки (3 тижні) при точці замовлення 18, якої вистачає лише на 3,6 тижня попиту.

Завдання 3:

Модель описує веб-службу з двома серверами. Запит надходить у пуассонівському потоці і з ймовірністю 0,7 направляється на головний сервер С1, з ймовірністю 0,3 – на допоміжний С2. Кожен сервер має чергу прийнятих запитів обмеженої довжини 1000: запит, що не вміщується, відхиляється (захист від DDOS). Програма сервера має пул потоків: запит з черги призначається вільному потоку і утримує його до кінця обробки. Потоку для обчислень потрібне процесорне ядро; потоків більше, ніж ядер, тому потік, готовий до обчислень, може чекати ядро. Після обчислень потік, звільнивши ядро, отримує з'єднання зі спільною для обох серверів базою даних (пул з'єднань – багатопотоковий доступ), записує результат, і лише після запису запит повертається користувачу, а потік і з'єднання звільняються. Час обробки запиту – від надходження до завершення запису в БД.

Таблиця 5 – Параметри моделі серверів

Параметр	**Значення**
Інтенсивність надходження запитів	200 запитів/с (інтервал exp(5) мс)
Ймовірність звернення до С2	0,3
Обмеження черги кожного сервера	1000
Потоків С1 / С2	32 / 16
Процесорних ядер С1 / С2	8 / 4
Час обчислень С1 / С2, мс	exp(40) / exp(50)
З'єднань з БД	10
Час запису в БД, мс	exp(20)

Параметри (табл. 5) обрано так, щоб система працювала в типовому режимі з запасом: завантаження ядер С1 $\lambda_1 \cdot E[S]/K = 140 \cdot 0{,}04 / 8 = 0{,}70$, ядер С2 – $60 \cdot 0{,}05 / 4 = 0{,}75$ (допоміжний сервер слабший: удвічі менше ядер і повільніші обчислення), з'єднань БД – $200 \cdot 0{,}02 / 10 = 0{,}40$. Час обчислень і запису прийнято експоненційним: так модель можна перевірити формулами теорії масового обслуговування. Одиниця часу моделі – мілісекунда.

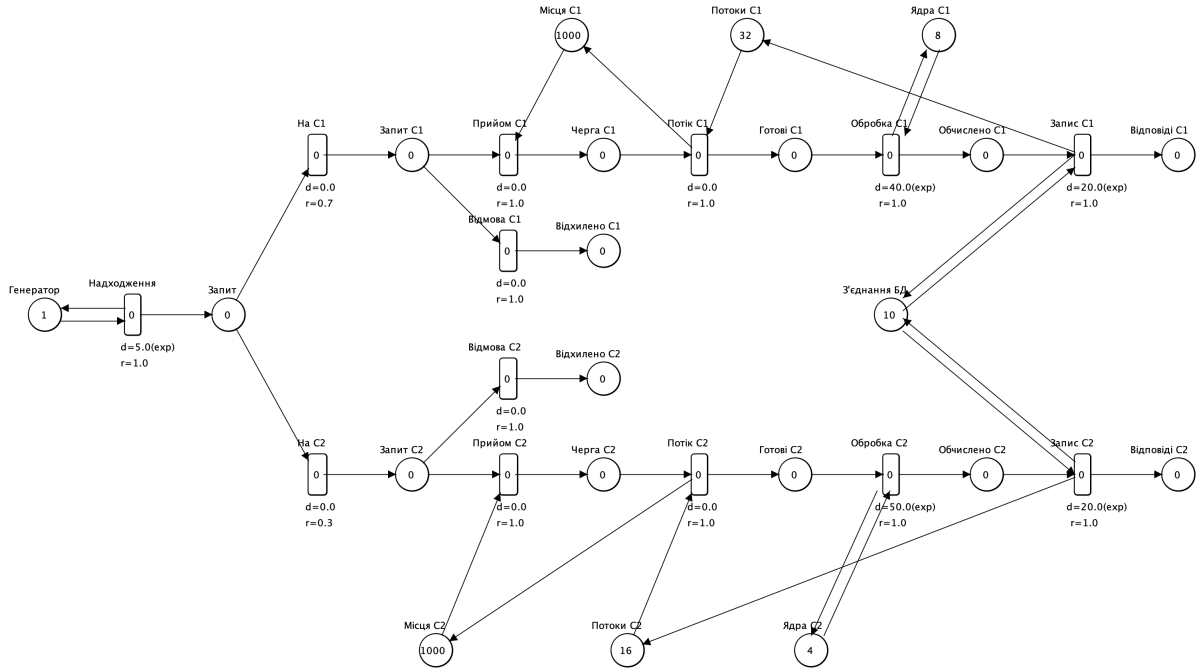


Рисунок 3 – Мережа Петрі моделі серверів зі спільною базою даних

Таблиця 6 – Переходи мережі серверів

Перехід	**Вхідні позиції**	**Вихідні позиції**	**Затримка**	**Пріоритет**	**Ймовірність**
Надходження	Генератор	Генератор, Запит	exp(5,0)	0	1,0
На С1	Запит	Запит С1	0	3	0,7
Прийом С1	Запит С1, Місця С1	Черга С1	0	2	1,0
Відмова С1	Запит С1	Відхилено С1	0	1	1,0
Потік С1	Черга С1, Потоки С1	Готові С1, Місця С1	0	0	1,0
Обробка С1	Готові С1, Ядра С1	Ядра С1, Обчислено С1	exp(40)	0	1,0
Запис С1	Обчислено С1, З'єднання БД	З'єднання БД, Потоки С1, Відповіді С1	exp(20)	0	1,0
На С2	Запит	Запит С2	0	3	0,3
Прийом С2	Запит С2, Місця С2	Черга С2	0	2	1,0
Відмова С2	Запит С2	Відхилено С2	0	1	1,0
Потік С2	Черга С2, Потоки С2	Готові С2, Місця С2	0	0	1,0
Обробка С2	Готові С2, Ядра С2	Ядра С2, Обчислено С2	exp(50)	0	1,0
Запис С2	Обчислено С2, З'єднання БД	З'єднання БД, Потоки С2, Відповіді С2	exp(20)	0	1,0

Мережа (рис. 3, табл. 6) має 21 позицію і 13 переходів. Ресурси подано позиціями з початковим маркуванням: Генератор – 1, З'єднання БД – 10, Місця С1 – 1000, Потоки С1 – 32, Ядра С1 – 8, Місця С2 – 1000, Потоки С2 – 16, Ядра С2 – 4. Перехід «Прийом Cs» (пріоритет 2) забирає маркер вільного місця черги; якщо

місць немає, запит забирає «Відмова Cs» (пріоритет 1). «Потік Cs» призначає запиту потік і повертає місце черги, «Обробка Cs» займає ядро на час обчислень, «Запис Cs» займає з'єднання БД і по завершенні повертає потік і з'єднання. Затримки «Обробка» і «Запис» – випадкові, решта переходів миттєві. Позиція «З'єднання БД» спільна для обох серверів.

Переходам розподілу «На С1» і «На С2» навмисно надано окремий пріоритет 3. Алгоритм `doConflikt` класу `PetriSim` розглядає як конфліктні всі переходи з найвищим пріоритетом, для яких виконано умову запуску, незалежно від того, чи мають вони спільні вхідні позиції, а переходу з ймовірністю 1 приписує ймовірність 1/i, де i – кількість таких переходів. У момент надходження запиту умову запуску має й перехід «Надходження» (маркер щойно повернувся в «Генератор»). Якщо всі ці переходи мають пріоритет 0, то з ймовірністю 1/3 першим вибирається «Надходження», а інакше накопичена сума 1/3 + 0,7 > 1, і завжди вибирається «На С1»; допоміжний сервер отримує лише 1/3 · 0,3 = 0,1 запитів. Імітація це підтвердила: при пріоритеті 0 частка запитів на С2 дорівнює 0,0998 ± 0,0010 замість 0,3, при окремому пріоритеті – 0,3004 ± 0,0007. У мережах завдань 1 і 2 така ситуація не виникає: там переходи з ймовірностями (0,8 / 0,2) мають пріоритет, який у той самий момент не має жоден інший дозволений перехід. Трасування перших подій:

```
t = 0.0000
  вхід у переходи:   Надходження
  маркування: З'єднання БД=10, Місця С1=1000, Потоки С1=32, Ядра С1=8, Місця С2=1000, Потоки
С2=16, Ядра С2=4
  у переходах: Надходження=1 (вихід 0.977)

t = 0.9770
  вихід з переходів: Надходження
  вхід у переходи:   На С2, Надходження
  маркування: З'єднання БД=10, Місця С1=1000, Потоки С1=32, Ядра С1=8, Місця С2=1000, Потоки
С2=16, Ядра С2=4
  у переходах: Надходження=1 (вихід 2.970), На С2=1 (вихід 0.977)

t = 0.9770
  вихід з переходів: На С2
  вхід у переходи:   Прийом С2
  маркування: З'єднання БД=10, Місця С1=1000, Потоки С1=32, Ядра С1=8, Місця С2=999, Потоки
С2=16, Ядра С2=4
  у переходах: Надходження=1 (вихід 2.970), Прийом С2=1 (вихід 0.977)

t = 0.9770
  вихід з переходів: Прийом С2
  вхід у переходи:   Потік С2
  маркування: З'єднання БД=10, Місця С1=1000, Потоки С1=32, Ядра С1=8, Місця С2=999, Потоки
С2=15, Ядра С2=4
  у переходах: Надходження=1 (вихід 2.970), Потік С2=1 (вихід 0.977)

t = 0.9770
  вихід з переходів: Потік С2
  вхід у переходи:   Обробка С2
  маркування: З'єднання БД=10, Місця С1=1000, Потоки С1=32, Ядра С1=8, Місця С2=1000, Потоки
С2=15, Ядра С2=3
  у переходах: Надходження=1 (вихід 2.970), Обробка С2=1 (вихід 121.903)
```

Миттєві переходи розподілу, прийому і призначення потоку спрацьовують послідовно в той самий момент, і запит одразу потрапляє на обчислення (ядра вільні); маркування ресурсів змінюється відповідно.

```
Перевірка інваріантів після кожної з 721514 подій: порушень немає
Середні значення бібліотеки і власні статистики прогону без розгону: найбільша розбіжність 6.26e-
10

РЕЗУЛЬТАТИ (20 прогонів по 600 с, розгін 20 с, 95 % довірчий інтервал):
  середній час відповіді (усі запити), мс        73.8964 ± 0.3304
  час відповіді (C1), мс                         64.5116 ± 0.1164
    очікування в черзі (C1), мс                  0.0009 ± 0.0007
    очікування ядра (C1), мс                     4.4709 ± 0.0668
    обчислення (C1), мс                          39.9985 ± 0.0538
    очікування з'єднання БД (C1), мс             0.0303 ± 0.0018
    запис у БД (C1), мс                          20.0110 ± 0.0336
  пропускна здатність (C1), запитів/с            139.9566 ± 0.2384
  завантаження ядер (C1)                         0.6998 ± 0.0016
  зайнято потоків (C1)                           9.0287 ± 0.0234
  середня довжина черги (C1)                     0.0001 ± 0.0001
  час відповіді (C2), мс                         95.7485 ± 1.0485
    очікування в черзі (C2), мс                  1.3223 ± 0.2786
    очікування ядра (C2), мс                     24.4840 ± 0.7317
    обчислення (C2), мс                          49.9287 ± 0.1109
    очікування з'єднання БД (C2), мс             0.0287 ± 0.0016
    запис у БД (C2), мс                          19.9849 ± 0.0520
  пропускна здатність (C2), запитів/с            60.0942 ± 0.1912
  завантаження ядер (C2)                         0.7501 ± 0.0032
  зайнято потоків (C2)                           5.6748 ± 0.0615
  середня довжина черги (C2)                     0.0796 ± 0.0169
  завантаження з'єднань БД                       0.4002 ± 0.0010
  частка запитів на C2                           0.3004 ± 0.0007
  частка відхилених запитів                      0.0000 ± 0.0000

Перевірка за мережею Джексона (M/M/K для ядер, M/M/10 для БД): час відповіді C1 64.5394, C2
95.5011, усі 73.8279 мс
Завантаження ядер C1: λ·E[S]/K = 0.7000
Завантаження ядер C2: λ·E[S]/K = 0.7500
Завантаження з'єднань БД: λ·E[S]/M = 0.4000
Сумарна пропускна здатність 200.0509 запитів/с при інтенсивності надходження 200.0

Переходи розподілу з пріоритетом 0 (як у переході «Надходження»): частка запитів на C2 0.0998 ±
0.0010 замість 0.3
```

Перевірка після кожної з 721514 подій контрольного прогону: для кожного сервера кількість вільних потоків разом із запитами, що їх утримують, дорівнює розміру пулу; вільні ядра разом з маркерами в «Обробка» – кількості ядер; вільні місця разом із чергою – 1000; вільні з'єднання разом із записами обох серверів – 10; кількість запитів, що надійшли, дорівнює сумі тих, що перебувають у системі, отримали відповідь або відхилені. Порушень немає.

Для аналітичної перевірки використано те, що за наявних параметрів потоків і місць у черзі практично завжди досить. Тоді ядра кожного сервера утворюють систему $M/M/K_s$, на яку надходить пуассонівський потік λ_s, а її вихідний потік за теоремою Берка теж пуассонівський, тож пул з'єднань БД – система M/M/10 з потоком $\lambda = \lambda_1 + \lambda_2$. Це мережа Джексона, і середній час відповіді

$$W_s = C(K_s, a_s) \cdot S_s / (K_s - a_s) + S_s + C(M, a_{БД}) \cdot S_{БД} / (M - a_{БД}) + S_{БД}, \quad a_s = \lambda_s S_s,$$

де C(c, a) – формула Ерланга C (ймовірність очікування). Аналітичні значення: W_1 = 64,54 мс, W_2 = 95,50 мс, W = 0,7W_1 + 0,3W_2 = 73,83 мс; імітація дала 64,51 ± 0,12, 95,75 ± 1,05 і 73,90 ± 0,33 мс. Завантаження ядер (0,700 і 0,750) і з'єднань БД (0,400) збігаються з $\lambda \cdot E[S]/K$ = 0,70; 0,75; 0,40, складові часу «обчислення» і «запис у БД» – із заданими середніми 40, 50 і 20 мс, а сумарна пропускна здатність – з інтенсивністю надходження. Модель верифіковано.

Оцінка часу обробки запиту за базових параметрів – 73,9 ± 0,3 мс. На головному сервері запит обробляється в середньому 64,5 мс, з яких 40 мс – обчислення, 20 мс – запис у БД і лише 4,5 мс – очікування ядра. На допоміжному – 95,7 мс: обчислення довші (50 мс), а через вище завантаження ядер очікування ядра становить 24,5 мс. Черги до потоків і до з'єднань БД практично немає, відхилених запитів немає.

Таблиця 7 – Вплив кількості потоків головного сервера (200 запитів/с)

Потоків C1	**Час відповіді C1, мс**	**Очікування в черзі, мс**	**Завантаження ядер**	**Зайнято потоків**	**Відхилено**
8	6001,0 ± 258,9	5941,1 ± 258,8	0,667 ± 0,002	8,00 ± 0,00	0,0262
10	78,8 ± 1,5	18,1 ± 1,4	0,701 ± 0,003	8,48 ± 0,04	0,0000
12	66,0 ± 0,5	4,3 ± 0,3	0,698 ± 0,004	8,62 ± 0,05	0,0000
14	64,7 ± 0,3	1,8 ± 0,1	0,700 ± 0,004	8,80 ± 0,05	0,0000
16	64,7 ± 0,4	0,9 ± 0,2	0,701 ± 0,004	8,94 ± 0,07	0,0000
24	64,6 ± 0,3	0,0 ± 0,0	0,704 ± 0,003	9,07 ± 0,04	0,0000
32	64,3 ± 0,5	0,0 ± 0,0	0,698 ± 0,005	9,00 ± 0,11	0,0000
64	64,4 ± 0,5	0,0 ± 0,0	0,698 ± 0,004	9,01 ± 0,09	0,0000

Експеримент з кількістю потоків головного сервера (табл. 7, 10 прогонів по 300 с) показує, що потоків має бути більше, ніж ядер. Потік утримується не лише під час обчислень, а й під час очікування ядра і запису в БД (у сумі ≈ 64 мс), тому пул з N потоків пропускає не більше N / 0,064 запитів/с. При 8 потоках (стільки ж, скільки ядер) це ≈ 124 запити/с < 140: усі потоки постійно зайняті, ядра простоюють (завантаження 0,667 замість 0,70), черга заповнюється до межі 1000, 2,6 % запитів відхиляється, а час відповіді зростає до 6,0 с. Уже 12 потоків дають час, близький до мінімального, а понад 16 потоків час відповіді не змінюється і збігається з аналітичним значенням для необмеженого пулу. Середня кількість зайнятих потоків C1 – близько 9.

Таблиця 8 – Час відповіді залежно від навантаження

Запитів/с	Потоки C1/C2	Час відповіді, мс	C1, мс	C2, мс	Теорія, мс	Відхилено	Макс. черга C1/C2
100	32/16	63,4 ± 0,3	60,0	71,3	63,5	0,0000	0 / 0
100	12/6	63,7 ± 0,2	60,2	72,1	63,5	0,0000	5 / 6
150	32/16	65,5 ± 0,2	60,7	76,7	65,6	0,0000	0 / 3
150	12/6	66,2 ± 0,3	61,1	78,2	65,6	0,0000	16 / 14
200	32/16	74,3 ± 0,6	64,4	97,3	73,8	0,0000	11 / 23
200	12/6	77,5 ± 0,7	66,1	104,0	73,8	0,0000	33 / 36
230	32/16	92,2 ± 2,3	72,2	138,7	90,9	0,0000	22 / 61
230	12/6	113,2 ± 3,8	79,8	191,1	90,9	0,0000	55 / 71
250	32/16	128,7 ± 4,3	85,6	229,4	132,8	0,0000	57 / 92
250	12/6	451,0 ± 143,9	120,4	1227,4	132,8	0,0000	124 / 419
280	32/16	2376,3 ± 329,0	279,7	7489,2	—	0,0019	226 / 1000
280	12/6	6695,0 ± 96,1	4738,9	11548,5	—	0,0444	1000 / 1000
400	32/16	7290,4 ± 17,0	5152,5	12617,7	—	0,2980	1000 / 1000
400	12/6	7727,7 ± 13,1	5436,5	13417,2	—	0,3481	1000 / 1000

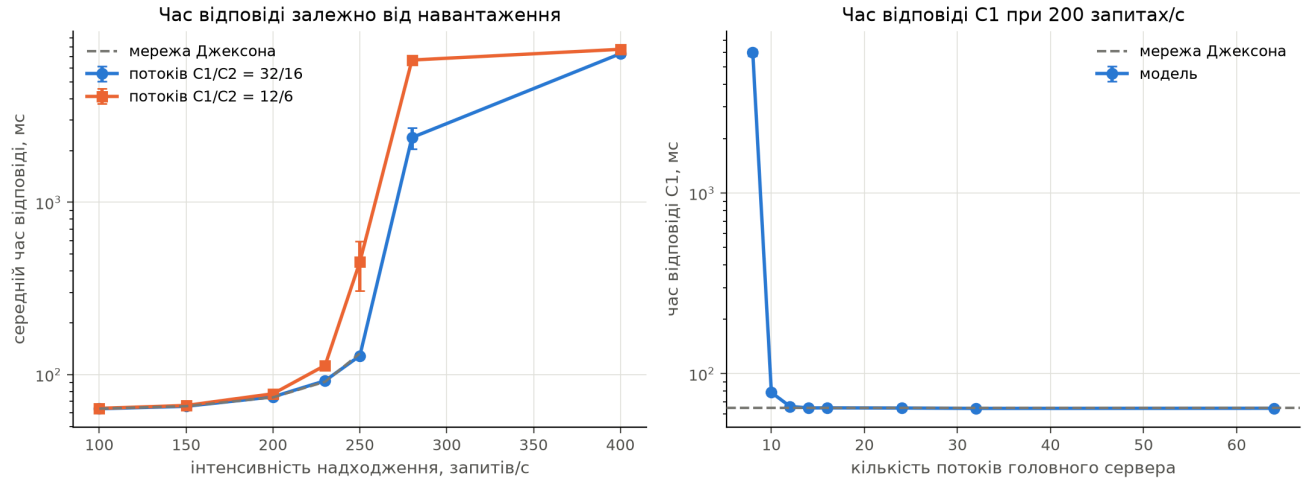


Рисунок 4 – Час відповіді залежно від навантаження і кількості потоків C1 (логарифмічна шкала)

Залежність від навантаження (табл. 8, рис. 4) порівнювалася для базових пулів потоків 32/16 і зменшених 12/6. До 230 запитів/с при достатній кількості потоків модель збігається з теорією, а час відповіді зростає повільно. Першим насичується допоміжний сервер: його ядра обробляють не більше 4 / 0,05 = 80 запитів/с, тобто 80 / 0,3 ≈ 267 запитів/с на вході; головний – 8 / 0,04 = 200, або ≈ 286 запитів/с. Тому при 280 запитах/с черга C2 досягає 1000, з'являються відмови, а час відповіді вимірюється секундами. Із меншими пулами потоків деградація

починається раніше, бо потоки, що чекають ядро або БД, стають вузьким місцем раніше за ядра.

Обмеження черги виконує свою функцію під час DDOS-навантаження 400 запитів/с: відхиляється 29,8 % запитів, а час відповіді обмежений – для прийнятих запитів він дорівнює часу проходження заповненої черги: 5,15 с на C1 (≈ 1000 / 200 запитів/с = 5 с плюс обробка) і 12,62 с на C2 (≈ 1000 / 80 = 12,5 с). Без обмеження черга і час відповіді зростали б необмежено.

Завдання 4:

Таблиця 9 – Пакет файлів мереж Петрі

Мережа	**Каталог**	**Файли**
Конвеєрна система (завдання 1)	nets/conveyor/	task.txt, conveyor.png, conveyor.pnml, conveyor_method.txt, conveyor.pns
Управління запасами (завдання 2)	nets/inventory/	task.txt, inventory.png, inventory.pnml, inventory_method.txt, inventory.pns
Сервери зі спільною БД (завдання 3)	nets/servers/	task.txt, servers.png, servers.pnml, servers_method.txt, servers.pns

Для кожної мережі сформовано пакет файлів (табл. 9) у каталозі `lab6/nets` репозиторію. Файли збережено функціями бібліотеки PetriObjModelPaint, якими користується графічний редактор, тому вони мають ті самі формати і відкриваються в редакторі:

- `task.txt` – текст завдання (для мережі серверів – також прийняті параметри);
- `<мережа>.pns` – графічна мережа `GraphPetriNet` з координатами елементів, збережена так само, як командою «Save Graph net» (серіалізація об'єкта графічної мережі);
- `<мережа>.png` – зображення цієї графічної мережі, намальоване методом `paintGraphPetriNet` класу `GraphPetriNet`, яким малює мережу полотно редактора (рис. 1–3);
- `<мережа>.pnml` – документ PNML, створений класом `PnmlModelGenerator` (команда «Save As…» редактора); мережу прочитано назад класом `PnmlModelParser`, і її склад збігся з вихідною для всіх трьох мереж;
- `<мережа>_method.txt` – java-метод `CreateNet…`, згенерований методом `saveNetAsMethod` класу `FileUse` (команда «Save net as method»), у форматі методів класу `NetLibrary`.

Початок і кінець файлу `servers_method.txt`:

```
public static PetriNet CreateNetServers() throws ExceptionInvalidNetStructure,
ExceptionInvalidTimeDelay {
    ArrayList<PetriP> d_P = new ArrayList<>();
    ArrayList<PetriT> d_T = new ArrayList<>();
    ArrayList<ArcIn> d_In = new ArrayList<>();
    ArrayList<ArcOut> d_Out = new ArrayList<>();
    d_P.add(new PetriP("Генератор",1));
    d_P.add(new PetriP("Запит",0));
    d_P.add(new PetriP("З'єднання БД",10));
    d_P.add(new PetriP("Запит С1",0));
    d_P.add(new PetriP("Місця С1",1000));
    d_P.add(new PetriP("Черга С1",0));
    d_P.add(new PetriP("Потоки С1",32));
    d_P.add(new PetriP("Готові С1",0));
    d_P.add(new PetriP("Ядра С1",8));
    d_P.add(new PetriP("Обчислено С1",0));
    d_P.add(new PetriP("Відповіді С1",0));
    d_P.add(new PetriP("Відхилено С1",0));
    d_P.add(new PetriP("Запит С2",0));
    ...
    d_Out.add(new ArcOut(d_T.get(12),d_P.get(15),1));
    d_Out.add(new ArcOut(d_T.get(12),d_P.get(19),1));
    PetriNet d_Net = new PetriNet("Servers",d_P,d_T,d_In,d_Out);
    PetriP.initNext();
    PetriT.initNext();
    ArcIn.initNext();
    ArcOut.initNext();

    return d_Net;
}
```

Завдання 5:

Стохастична мережа Петрі з часовими затримками описується кортежем

$$N = (P, T, A, W, I, K, R, \tau, \boldsymbol{M}_0),$$

де $P = \{P_1, \ldots, P_{21}\}$ – позиції, $T = \{T_1, \ldots, T_{13}\}$ – переходи, A – дуги, $W(P_i, T_j)$ і $W(T_j, P_i)$ – кратності вхідних і вихідних дуг (усі рівні 1), I – інформаційні дуги (у цій мережі їх немає), K_j – пріоритет, R_j – ймовірність спрацювання, τ_j – випадкова затримка переходу, M_0 – початкове маркування. Позначення позицій і переходів відповідають порядку елементів у файлі `servers_method.txt`:

Таблиця 10 – Позиції мережі серверів

	Позиція	**M_0**		**Позиція**	**M_0**
P_1	Генератор	1	P_{12}	Відхилено С1	0
P_2	Запит	0	P_{13}	Запит С2	0
P_3	З'єднання БД	10	P_{14}	Місця С2	1000
P_4	Запит С1	0	P_{15}	Черга С2	0
P_5	Місця С1	1000	P_{16}	Потоки С2	16
P_6	Черга С1	0	P_{17}	Готові С2	0
P_7	Потоки С1	32	P_{18}	Ядра С2	4
P_8	Готові С1	0	P_{19}	Обчислено С2	0
P_9	Ядра С1	8	P_{20}	Відповіді С2	0
P_{10}	Обчислено С1	0	P_{21}	Відхилено С2	0
P_{11}	Відповіді С1	0			

Стан мережі в момент t задається вектором маркування $M(t) = (M_1(t), \ldots, M_{21}(t))$ і для кожного переходу T_j масивом моментів виходу маркерів з його каналів $E_j(t) = \{e_{j1}, e_{j2}, \ldots\}$; кількість зайнятих каналів $b_j(t) = |E_j(t)|$ (перехід багатоканальний, канали не обмежені). Умова запуску переходу:

$$Z_j(\boldsymbol{M}) = \prod_{P_i \in \bullet T_j} [M_i \geq W(P_i, T_j)] \cdot \prod_{(P_i, T_j) \in I} [M_i \geq W(P_i, T_j)],$$

де $[\cdot]$ дорівнює 1, якщо умова виконана, і 0 інакше. Множина переходів, що можуть запуститись, і розв'язання конфлікту:

$$D(\boldsymbol{M}) = \{T_j : Z_j(\boldsymbol{M}) = 1\}, \quad D^*(\boldsymbol{M}) = \{T_j \in D : K_j = \max_{T_k \in D} K_k\}, \quad P\{T_j\} = R_j, \ T_j \in D^*.$$

У класі `PetriSim` перехід з $R_j = 1$ у множині D* отримує ймовірність $1/|D^*|$; у мережі серверів ймовірності задано лише переходам T_2 і T_8, і тільки вони мають пріоритет 3.

Вхід маркерів у вибраний перехід T_j у момент t_n (повторюється, поки $D(M) \neq \varnothing$):

$$M_i := M_i - W(P_i, T_j), \ P_i \in \bullet T_j; \quad E_j := E_j \cup \{t_n + \tau_j\}.$$

Просування часу до найближчої події і вихід маркерів з усіх каналів, момент виходу яких настав:

$$t_{n+1} = \min_j \min E_j; \quad \text{для } e \in E_j, e = t_{n+1}: M_i := M_i + W(T_j, P_i), \ P_i \in T_j\bullet; \quad E_j := E_j \setminus \{e\}.$$

Разом зміна маркування між двома подіями записується рівнянням

$$\boldsymbol{M}(t_{n+1}) = \boldsymbol{M}(t_n) - \Sigma_j\, X_j(t_n)\, \boldsymbol{W}^-_j + \Sigma_j\, Y_j(t_{n+1})\, \boldsymbol{W}^+_j,$$

де $X_j(t_n)$ – кількість входів маркерів у перехід T_j у момент t_n, $Y_j(t_{n+1})$ – кількість виходів з нього в момент t_{n+1}, W^-_j, W^+_j – вектори кратностей вхідних і вихідних дуг переходу. Для мережі серверів (s = 1 – головний сервер, s = 2 – допоміжний) ці рівняння мають вигляд, наведений у таблиці. Для С1 використовуються позиції P_4–P_{12}, для С2 – P_{13}–P_{21} у тому самому порядку.

Таблиця 11 – Рівняння переходів мережі серверів

Перехід	Умова запуску $Z_j = 1$	Вхід маркерів	Вихід маркерів	τ_j, мс	K_j	R_j
T_1 Надходження	$M_1 \geq 1$	$M_1 - 1$	$M_1 + 1, M_2 + 1$	Exp(5)	0	1
T_2 На С1	$M_2 \geq 1$	$M_2 - 1$	$M_4 + 1$	0	3	0,7
T_3 Прийом С1	$M_4 \geq 1, M_5 \geq 1$	$M_4 - 1, M_5 - 1$	$M_6 + 1$	0	2	1
T_4 Відмова С1	$M_4 \geq 1$ (фактично при $M_5 = 0$)	$M_4 - 1$	$M_{12} + 1$	0	1	1
T_5 Потік С1	$M_6 \geq 1, M_7 \geq 1$	$M_6 - 1, M_7 - 1$	$M_8 + 1, M_5 + 1$	0	0	1
T_6 Обробка С1	$M_8 \geq 1, M_9 \geq 1$	$M_8 - 1, M_9 - 1$	$M_9 + 1, M_{10} + 1$	Exp(40)	0	1
T_7 Запис С1	$M_{10} \geq 1, M_3 \geq 1$	$M_{10} - 1, M_3 - 1$	$M_3 + 1, M_7 + 1, M_{11} + 1$	Exp(20)	0	1
T_8 На С2	$M_2 \geq 1$	$M_2 - 1$	$M_{13} + 1$	0	3	0,3
T_9 Прийом С2	$M_{13} \geq 1, M_{14} \geq 1$	$M_{13} - 1, M_{14} - 1$	$M_{15} + 1$	0	2	1
T_{10} Відмова С2	$M_{13} \geq 1$ (фактично при $M_{14} = 0$)	$M_{13} - 1$	$M_{21} + 1$	0	1	1
T_{11} Потік С2	$M_{15} \geq 1, M_{16} \geq 1$	$M_{15} - 1, M_{16} - 1$	$M_{17} + 1, M_{14} + 1$	0	0	1
T_{12} Обробка С2	$M_{17} \geq 1, M_{18} \geq 1$	$M_{17} - 1, M_{18} - 1$	$M_{18} + 1, M_{19} + 1$	Exp(50)	0	1
T_{13} Запис С2	$M_{19} \geq 1, M_3 \geq 1$	$M_{19} - 1, M_3 - 1$	$M_3 + 1, M_{16} + 1, M_{20} + 1$	Exp(20)	0	1

У табл. 11 вхід маркерів супроводжується додаванням моменту $t_n + \tau_j$ до масиву E_j, а вихід – вилученням моменту t_{n+1} з нього. Конфлікт за маркер у P_2 розв'язується лише між T_2 і T_8, бо тільки вони мають пріоритет 3; конфлікт за маркер у P_4 (P_{13}) – на користь T_3 (T_9) за пріоритетом, тому T_4 (T_{10}) спрацьовує тільки при $M_5 = 0$ ($M_{14} = 0$). Конфлікти за ресурси P_3, P_7, P_9 розв'язуються між переходами з однаковим пріоритетом 0 випадковим вибором.

З рівнянь переходів випливають Р-інваріанти мережі, які виконуються в будь-який момент часу і перевірялися програмою після кожної події:

$$M_1 + b_1 = 1; \quad M_3 + b_7 + b_{13} = 10; \quad M_9 + b_6 = 8; \quad M_{18} + b_{12} = 4;$$

$$M_5 + b_3 + M_6 + b_5 = 1000; \quad M_7 + b_5 + M_8 + b_6 + M_{10} + b_7 = 32$$ (аналогічно для С2 з 1000 і 16).

Вихідні характеристики моделі визначаються через середні за часом значення маркування позицій і кількості зайнятих каналів переходів на інтервалі

збору статистики $[T_0, T]$:

$$\overline{M_i} = (1 / (T - T_0)) \int_{T_0}^{T} M_i(t)\, dt, \quad \overline{b_j} = (1 / (T - T_0)) \int_{T_0}^{T} b_j(t)\, dt;$$

бібліотека обчислює їх рекурентно (для $T_0 = 0$) за формулою $\bar{M}(t_{n+1}) = \bar{M}(t_n) + (M(t_n) - \bar{M}(t_n))\cdot(t_{n+1} - t_n) / t_{n+1}$. Середній час обробки запиту на сервері s – за формулою Літтла:

$$\lambda_1 = Y_7 / (T - T_0), \quad W_1 = (\overline{M_6} + \overline{M_8} + \overline{b_6} + \overline{M_{10}} + \overline{b_7}) / \lambda_1, \quad W = (L_1 + L_2) / (\lambda_1 + \lambda_2),$$

де Y_7 – кількість виходів з переходу T_7 (відповідей C1) за час збору статистики, L_s – чисельник W_s; для C2 використовуються P_{15}, P_{17}, T_{12}, P_{19}, T_{13}. Кожен доданок, поділений на λ_s, дає складову часу: очікування в черзі, очікування ядра, обчислення, очікування з'єднання БД і запис. Завантаження ядер $\rho_1 = \bar{b}_6 / 8$, $\rho_2 = \bar{b}_{12} / 4$, з'єднань БД – $(\bar{b}_7 + \bar{b}_{13}) / 10$, частка відхилених запитів – $(Y_4 + Y_{10}) / Y_1$.

Висновок

У роботі вивчено бібліотеку PetriObjModelPaint і графічний редактор мереж Петрі та з використанням алгоритму імітації класу PetriSim реалізовано три моделі: конвеєрну систему і систему управління запасами з практикуму 5 та модель двох серверів зі спільною базою даних.

Мережі завдань 1 і 2 побудовано класами бібліотеки з тими самими позиціями, переходами, затримками, пріоритетами, ймовірностями та дугами, що й у практикумі 5, і програмна перевірка підтвердила збіг структури. Верифікацію виконано трасуванням подій, перевіркою P-інваріантів після кожної події, балансовими співвідношеннями і порівнянням з незалежною реалізацією практикуму 5: результати збігаються в межах 95 % довірчих інтервалів. Конвеєр працює на межі можливостей: у середньому зайнято 4,01 пристрої з 5, завантаження спадає від 0,88 до 0,69 уздовж конвеєра, а середній час перебування деталі 9,7 хв визначається переважно рухом конвеєром і поверненнями. У системі запасів у середньому 28,8 холодильника, продажі втрачаються в середньому раз на 2,33 тижня, причому серіями наприкінці циклів замовлення.

Для задачі про сервери побудовано мережу, у якій ресурси (місця в черзі, потоки, процесорні ядра, з'єднання з БД) подано позиціями з маркерами, а багатоканальність переходів дає змогу моделювати одночасну обробку багатьох

запитів. Модель верифіковано інваріантами і мережею Джексона (M/M/K для ядер, M/M/10 для БД): розбіжність з аналітичним часом відповіді менша за довірчий інтервал. За прийнятих параметрів середній час обробки запиту становить 73,9 ± 0,3 мс (64,5 мс на головному і 95,7 мс на допоміжному сервері). Потоків має бути помітно більше, ніж ядер, бо потік утримується і під час запису в БД: при 8 потоках на 8 ядер головний сервер не справляється з навантаженням, а від 12–16 потоків час відповіді вже мінімальний. Пропускну здатність системи обмежує допоміжний сервер (≈ 267 запитів/с на вході), а обмеження черги 1000 під час перевантаження обмежує час відповіді кількома секундами ціною відхилення частини запитів.

Для кожної мережі сформовано пакет файлів: текст завдання, графічну мережу та її зображення, документ PNML і java-метод, збережені функціями бібліотеки; мережі, прочитані з PNML, збігаються з вихідними. Для мережі серверів записано рівняння стохастичної мережі Петрі: умови запуску, розв'язання конфліктів, зміну маркування і масивів моментів виходу при вході та виході маркерів, просування часу, P-інваріанти і формули вихідних характеристик.

Під час роботи з PetriSim виявлено особливості, які треба враховувати. Маркери неіменовані, тому час перебування вимоги оцінюється за формулою Літтла, а не вимірюється для кожної вимоги. Метод `go()` класу PetriSim зупиняє мережу з генератором після першої події, а середні в методі `step()` обчислюються з вагою $\Delta t / T_{мод}$, тому імітацію слід виконувати за схемою `PetriObjModel.go` (input – просування часу – output – input), а статистику з розгоном збирати окремо. Розв'язання конфліктів враховує всі дозволені переходи з найвищим пріоритетом, а не лише ті, що конкурують за маркер, тож переходам вибору за ймовірністю потрібен окремий пріоритет: інакше частка запитів на допоміжний сервер падає з 0,3 до 0,1. Генератор випадкових чисел бібліотеки (`Math.random`) не має зерна, тому прогони незалежні, але не відтворювані, а точність оцінок забезпечується серіями прогонів з довірчими інтервалами.